From Sparse Selection to Physical Savings: A Unified Runtime and Measurement Contract for Progressive Retrieval Attention

PRA Research Program

August 2026

Abstract

Progressive Retrieval Attention (PRA) lets a query select a bounded subset of URI-addressed, model-native key/value memory. Earlier work established the model interface, external-memory lifecycle, typed capability records, and compact typed-result backing, but those parts did not yet form one product surface, and logical sparsity did not by itself establish lower latency or accelerator memory. We introduce a unified runtime that preserves the existing Hugging Face API while adding an engine-neutral four-axis execution policy, request-owned selection plans, versioned systems configuration, authenticated sessions, lazy tool and skill disclosure, independently authorized tool execution, session-scoped result compaction, size-adaptive native result indexing, lazy selected-region native encoding, durable task-aware agent state, selective replay, deduplicated K/V interval planning, physical byte accounting, a versioned cross-model product-profile registry, bounded caching, profiling, and a scheduler-unaware serving handoff. A standalone gateway now separates agent semantics, PRA mediation, and engine-native state through a JSON logical-reference contract and explicit capability negotiation. The agent now constructs an OpenAI-style conversational spine plus detached typed records; AUTO negotiation preserves those records for PRA-aware gateways and engines and renders text only at a compatibility boundary. It streams HF-backed responses with request-owned cleanup. The gateway now resolves engine-specific cache/session profiles, exercises prepare/close lifecycles, preserves append-only G10 prefixes, and transports message and resource deltas without conflating conventional prefix K/V with PRA-native state. A five-turn deterministic profile preserves the prior serialized prefix on all four eligible turns; physical cache hits remain unmeasured. A native-reference audit found and corrected a positional collision: external keys retained source-relative RoPE coordinates while direct queries restarted at zero. On a random tiny Qwen fixture, the corrected full-reference path matches visible-prefix logits to 5.97×10^{-8} maximum absolute error. On three one-token continuation cases, full native replay preserves visible-prefix top tokens and answer accuracy for Qwen3-0.6B and a public Llama-3.2-1B weight mirror. Gemma-3-1B preserves disabled behavior, native MQA shape, positions, and decode lifetime, but its four global PRA consumers do not reproduce the semantics of a visible prefix passing through 22 unchanged local sliding layers; we report that family as partial topology. The runtime now separates address encoding, detail-K/V encoding, request-time routing, and memory consumption. A corrected 102-row layer calibration finds that no sparse profile is portable across the three families: all eligible layers are the balanced correctness profile for each model, while Qwen's last-24 profile is only a small-cohort quality-max candidate. A 48 GiB M4 Pro calibration then extends Qwen to 8B, 14B, 32B, and 30B-A3B over 60 unique natural-QA model–example pairs. Full concatenated native replay preserves every selected-text sequence. Dense 32B is less likelihood-sensitive to reduced consumer suffixes, but no sparse profile preserves enough sequence identity to graduate beyond calibration-pending. DeepSeek Harness and Pi bridges preserve typed tool-result identity through explicit ordinary-engine fallback in all 10 contract cases. In a five-seed tiny-HF mechanism profile, token-shared selection required 2.8 routing operations per request, versus 8.4 for token-per-layer, a 3.0× reduction with three active layers. On a CUDA 12.6 host with an NVIDIA GTX 950M, gathering 512 of 8,192 native-K/V tokens takes a median 0.253 ms at batch one, and packing eight selected intervals takes 0.415 ms. The same packed payload costs 0.424 ms when HBM-resident, 1.592 ms from pinned CPU memory, and 2.157 ms from ordinary CPU memory. Thus, off-device transfer is 3.75–5.08 times the HBM-resident packing cost on this host. Compilation is a negative capability gate: the CPU toolchain lacks an available compiler and the GPU is below the Triton capability floor. No optional serving engine is installed. The result is therefore a tested portable eager baseline and reproducible measurement contract, not a fused-kernel or serving-engine speed claim. Companion Apple-M5 runs apply that contract to one frozen 60-example selection cohort in MLX-LM, SGLang-MLX, and vLLM-Metal. Native E2 removes 90.6–93.0% of visible prompt tokens, and all three engines preserve all 840 paired outputs. The corrected vLLM run first restores E0 source pages through APC and captures E2 source pages from vLLM's own prefill, preventing fused-kernel geometry from being confounded with representation. An expanded 149-question confirmation per engine preserves 6,258/6,258 pairs overall. Four-regime cost ratios still show that semantic transport does not by itself imply a large economic win. A separate live storage study connects the same lifecycle manager to vLLM pages, SGLang HiCache-backed arrays, and MLX arrays. Across 17 model/dataset cohorts spanning Qwen, Llama, and Gemma, lossless WARM preserves 265/265 outputs and every manager recovers after restart; int8 COLD is exact in only 61/265 sequences and remains opt-in. Gemma is unsupported by the pinned SGLang-MLX backend because that runtime cannot represent its per-layer sliding-window topology.

1 Introduction

PRA changes what a model must keep logically available. A long-lived resource may remain addressable through a compact routing representation while only a small selected part of its native K/V participates in one attention operation. That distinction is algorithmically useful, but it leaves a systems question unanswered. A selected set can be small while the runtime still stores every detail tensor in HBM, launches many tiny gathers, copies the same bytes repeatedly, constructs temporary concatenations, or serializes requests around Python control flow. In that case sparsity exists in a trace but not in the resource bill.

This paper asks how to turn PRA into a portable inference primitive before a serving engine becomes semantically aware of PRA memory. The immediate task is not to design a new scheduler. It is to define an execution boundary that can be measured, optimized, and later replaced without changing the meaning of a selected reference.

Three previously separate interfaces must meet at that boundary. The Paper 2 SDK supplies `PRAForCausalLM`, model-family injection, routing, and generation. Paper 4 supplies authenticated resolvers and the cold, warm, and hot external-memory lifecycle. Paper 6.5 supplies stable typed identities, callable-derived tool schemas, declarative skills, lazy disclosure, and host-authorized tool execution. Paper 7 supplies type-aware compact views, scoped lossless backing, size-gated native indexing, lazy selected-region encoding, selective materialization, and cursors for large results. Paper 8 supplies versioned task state, task-scoped record admission, and durable user/session resolution. We retain these responsibilities and place one orchestration facade above them. This is integration by contract rather than by collapsing distinct policies.

The contributions are:

- a unified runtime contract joining model memory, authenticated resources, typed capabilities and results, durable tasks and sessions, and independently authorized tool execution without collapsing their policy boundaries;
- a physical K/V contract that preserves grouped-query heads, separates selection sparsity, materialization width, and consumer-layer coverage, merges intervals before budgeting, and accounts for transfers, temporary bytes, and bounded cache reuse;

- a corrected source-relative position and request-lifetime implementation, with exact disabled behavior and measured full-native semantic gates on Qwen and Llama plus a localized Gemma global/local topology limitation;
- an engine-neutral, engine-type-aware gateway with independent PRA and ordinary cache/session capabilities, explicit lifecycle, append-only G10 placement, message/resource deltas, affinity hints, and non-sensitive session inspection, while advertising phase selection separately from unsupported scheduler hints;
- lazy tools and skills, size-adaptive typed-result indexing, task-scoped acquisition, external-agent bridges, and a user-facing runtime/agent interface; and
- reproducible eager measurements and explicit negative gates for compilation, modern-GPU transfer overlap, and native serving-engine integration.

2 The Systems Gap

2.1 Logical and physical memory are different quantities

Consider L decoder layers. Each reference has T tokens, H_{kv} physical key/value heads, head width D, and b bytes per scalar. Storing complete K/V for the reference requires

$$B_{\text{detail}} = 2LH_{kv}TDb, \tag{1}$$

where the factor two accounts for keys and values. If routing selects S unique tokens, the active payload is

$$B_{\text{active}} = 2L_cH_{kv}SDb, \tag{2}$$

where L_c is the number of memory-consuming layers. The ratio $B_{\text{active}}/B_{\text{detail}}$ is a logical opportunity. It is not peak HBM. Peak HBM also includes model weights, local K/V, routing indices, allocator slack, transfer staging, and temporary packed tensors.

Grouped-query and multi-query attention make the distinction especially important. Warm storage should preserve H_{kv} physical heads. Expanding them to the larger number of query heads before materialization duplicates memory without adding information. The runtime therefore represents every native tensor as

$$K, V \in \mathbb{R}^{B\times H_{kv}\times T\times D} \tag{3}$$

and leaves head expansion to the model-family attention adapter.

These equations require four layer sets, not one overloaded set of "PRA layers." Let L_A store compact addresses, L_{KV} store detailed native K/V, L_R perform request-time routing, and L_C consume selected memory. The runtime enforces

$$L_C \subseteq L_{KV}, \qquad L_R \subseteq L_A \quad \text{for layer-native routing.} \tag{4}$$

The persistent detail term is therefore

$$B_{\text{detail}} = 2|L_{KV}|H_{kv}TDb, \tag{5}$$

whereas one request pays active attention over $|L_C|S$ layer-token states. Address layers may be broader than detail-K/V layers when compact search is useful at several depths. Production defaults do the opposite kind of conservatism: detail K/V is encoded only for the resolved consumer profile, unless profile switching or calibration explicitly requests a union.

2.2 Latency is a sum of visible stages

For one request, PRA overhead is not a single routing timer. We decompose it as

$$t_{\text{PRA}} = t_{\text{lookup}} + t_{\text{graph}} + t_{\text{plan}} + t_{\text{gather}} + t_{\text{transfer}} + t_{\text{pack}} + t_{\text{position}} + t_{\text{metadata}}. \tag{6}$$

Generation and attention follow this preparation. Cold requests also pay source fetch and native encoding; compiled paths pay graph capture and code generation. Reporting only warm attention can therefore reverse the practical conclusion.

3 A Unified Runtime Contract

3.1 One facade, separate authorities

Figure 1 shows the runtime. The facade does not make every component interchangeable in meaning. It gives them one lifecycle and one inspection surface while retaining their ownership boundaries.

Agent / existing harness / CLI

↓

PRA gateway: logical IDs, capability negotiation, protocol trace

G00 pass-through   G10 text fallback   G01 upgrade   G11 mediation

↓ JSON logical request; no raw K/V ↓

Inference runtime: config, sessions, traces, cache accounting

Model path	**Resource path**	**Tool path**
PRAForCausalLM	ExternalMemoryManager	ResourceDiscoveryEngine
route → select	cold → warm → hot	discover → disclose
native K/V consume	resolver + scoped caches	SafeToolExecutor

lazy tool/skill palette → exact capability view

compact result → scoped backing → selected replay / cursor

↓

deduplicated plan → gather/transfer/pack → HF or thin engine handoff

Figure 1: The distributed product boundary. Stable URIs join the paths, but the agent owns task meaning, the gateway owns translation, and the engine owns native K/V. Routing does not authorize source access or tool side effects.

The public configuration has two layers. `PRAConfig` specifies semantic behavior and all four model-layer roles: `address_layers`, `detail_kv_layers`, `routing_layers`, and `consumption_layers`, plus chunking, closure mode, and materialization budget. `PRARuntimeConfig` specifies physical behavior: backend, compilation mode, K/V layout, page size, cache bounds, prefetch policy, and timing synchronization. Both serialize to a versioned JSON artifact. A runtime optimization is valid only when changing the second layer preserves selected identities and output parity under fixed model semantics.

Layer selections use topology-normalized forms: all layers, last n, last fraction, evenly spaced n, or an explicit list. A versioned registry resolves request override, workload/model profile, family balanced fallback, then the all-eligible correctness reference. The named objectives are `reference_correctness`, `quality_max`, `balanced`, and `economy`; the resolved trace records model ID and revision, workload, materialization class, physical layer lists, profile source, and registry version. These are internal optimization objectives. The product registry labels smoke-only realizations of `quality_max` as `QUALITY_MAX_CANDIDATE`. A profile name is not portable evidence by itself.

Address and detail construction have independent lifecycle states (`NOT_BUILT`, `PARTIAL`, `BUILT`, `SKIPPED`, or `DEFERRED`). The detail policy is minimal, profile union, all layers, or explicit. The address policy is routing-only, all candidate routing layers, external-only, or explicit. If a new consumer requires absent detail K/V, the runtime must re-encode missing layers, fail, or follow an explicitly configured profile downgrade. It never silently substitutes a shallower profile.

3.2 A multi-axis execution policy

PRA does not imply one routing frequency. We represent one request with four orthogonal choices. The *selection stage* is request, prefill/completion phase, or token. The *layer scope* is shared logical identities or an independent selection per active layer. The *materialization scope* is request, phase, layer, or token. The *residency policy* retains a payload for that lifetime or releases it after one layer window. Shared identities never mean shared tensors: chunk C_7 resolves to the native K/V independently encoded at each consuming layer.

The global default is request/shared/request/keep. A model default can override individual fields, and a request override takes final precedence. The resolved policy records field-level provenance. Unsupported combinations raise before generation; there is no implicit downgrade. The engine-neutral `PRASelectionPlan` contains stable URI/chunk identities and row-local spans, never tensors. A layer resolver maps that plan to native payloads, and a separate materialization manager applies physical lifetime and residency.

The HF reference path preserves the old high-level request/shared behavior and the old low-level token/per-layer behavior. It adds request/per-layer by capturing all active layers in one memory-disabled prompt pass, routing each once, and freezing each result. It adds token/shared through a request-owned bridge: layers before the designated routing layer receive no current-token PRA memory; that layer selects once, and later layers resolve the same identities to their own K/V. Phase/shared execution uses a cache-free routing probe at the first active PRA layer, seeds the shared logical plan, and then performs one cache-producing prefill. Decode reselects once at the same boundary and reuses the resulting plan. Requiring the routing layer to be the first active consumer prevents an earlier layer from entering the cache with a different memory view.

3.3 Stable resources and authenticated sessions

A stable URI is the common identity for text memory, native K/V, tools, and typed observations. Paper 4's `PRASession` retains tenant, user, session, cache scope, admitted resources, and warm/hot handles. Credentials remain opaque callbacks passed only to resolvers; they are absent from prompts, snapshots, and artifacts. Cache presence never grants authorization.

3.4 Durable logical sessions and task scope

An interactive agent needs continuity longer than one model process. The runtime therefore separates logical `AgentSessionState` from physical `PRASession`. Logical state is resolved by user and session identity and contains an ordered typed-record stream, a task descriptor, and an idempotent event log. Physical state owns admitted resources, native K/V, and result-store handles. Closing it releases accelerator and resolver state without deleting the conversation or task graph.

`SessionService` is the persistence boundary. Its in-memory implementation provides thread-safe ephemeral operation. Its local implementation hashes path identities, writes versioned JSON through atomic replacement, and rejects stale optimistic versions. A database can implement the same create, resolve, save, list, and delete contract without changing the model path.

Task events create, link, activate, block, resume, complete, or cancel nodes in a validated DAG. Typed records carry producing-task, consuming-task, relation, and event-sequence provenance. Before ordinary PRA ranking, `TaskScopeSelector` admits the active task, its structural closure, or a bounded adaptive neighborhood. Scope controls which records may compete; record and chunk routing still control which admitted detail is materialized.

3.5 Agent and toolset product surface

`Toolset` couples a callable to its generated schema, namespace, tenant, tags, and declared side-effect class. Bundles compose through the existing `CapabilitySDK`. The built-in local bundle provides file listing, bounded reads and search, exact writes and replacements, Git status, and workspace-rooted command execution. Path operations reject traversal outside the configured workspace. Shell commands remain write-class actions: workspace selection is not a security sandbox.

`PRAAgent` assembles model, capability discovery, lazy skill folders, safe execution, compact results, durable sessions, and task scope. A turn persists the user record, selects task-relevant records, discloses a bounded tool palette, generates, validates at most a configured number of tool rounds, stores compact output with exact-backing provenance, and persists the final assistant record. The terminal adds resumable sessions and slash commands for tasks, context, and tools. Write and destructive calls are denied by default and can receive one host confirmation; disclosure alone never grants authority.

Paper 6.5's tool resources enter through the same runtime but a separate decision path. Discovery can identify a tool. Disclosure can expose its schema. A model can propose a call. Only `SafeToolExecutor`, given a request-scoped `ExecutionAuthorization`, can validate identity, arguments, write authority, and destructive authority. Its result becomes a typed observation resource with producer and call provenance. This separation prevents a high retrieval score from turning into ambient execution privilege.

3.6 Lazy tools and skills

Paper 6.5's latest capability contract is now part of the facade rather than an adjacent experiment. `AgentConfig` accepts ordinary Python callables, explicit declarative `Skill` objects, and a parent directory containing OpenAI- or Anthropic-style skill folders. Callables are inspected into stable, provider-neutral schemas. Folder loaders parse declarative metadata and instructions but never execute bundled scripts.

The default is a two-stage model-side lifecycle. Discovery and Phase A encode only compact selection views under count and token budgets. Once the model or host chooses a stable record ID, `activate_capability` activates the complete tool schema or skill instructions from immutable local backing. It does not rerun semantic discovery. Encoded views may be cached, while active bytes and resident cached bytes remain separate counters. Eager selection or full-view encoding is an explicit policy override.

3.7 Compact typed result backing

A successful tool output need not return as one flat prompt block. Every open `PRASession` now owns an `AdaptiveContextRuntime` with an exact, hash-verified backing store scoped by tenant and session. The runtime infers tabular, log, graph, terminal, or generic structured shape for tool and API results. Type-specific compactors retain schema, counts, statistics, errors, and bounded representative units while lexical, entity, rare-term, schema, and optional summary addresses remain retrieval-only.

The public operations preserve stable record identity: compact inspection, address search, full or selected materialization, and bounded cursor actions. `execute_tool_and_record` joins authorized execution to this record lifecycle while retaining producer, call, and observation provenance. Closing an ephemeral session removes its backing bytes.

Paper 7's final result changes the product default. Native PRA is appropriate for typed backing only while its ingestion and resident-state cost stays inside an explicit budget. Above that budget, the record must remain recoverable through cheaper addresses, search, cursors, and bounded lazy native encoding; it must not be truncated and labeled as a complete native index.

Accordingly, `ContextPolicy` exposes `max_native_index_tokens` and `max_native_index_bytes`, plus per-`RecordType` overrides. The byte gate can reject an oversized descriptor before tokenization; the token gate more directly bounds model work when token accounting is available. The reference defaults are 4,096 tokens and 65,536 bytes, but these are deployment policy values rather than claimed universal optima.

Every record has one auditable native-index lifecycle state:

`NOT_REQUESTED  BUILT  SKIPPED_SIZE_LIMIT  DEFERRED`

The ordinary Hugging Face loader automatically prepares in-budget records. `BUILT` means complete backing passed through the production PRA encoder. `SKIPPED_SIZE_LIMIT` preserves compact and cheap-index recovery. `DEFERRED` is an explicit synchronous lifecycle decision, not a claim of background work. After a cheap selector identifies a bounded row, field, line, item, or chunk, `encode_result_region_native` authorizes that exact region and creates native K/V only for it. Teardown removes compact, full, and lazy-region references. A shared multi-tenant model cache is not claimed safe without engine-level partitioning.

3.8 User workflow

The supported workflow is deliberately small:

```
agent = AgentConfig(tools=(lookup,), skills_path="./skills")
context = ContextPolicy(
    max_native_index_tokens=4096,
    max_native_index_bytes=65536,
    record_policies={DB_RESULT: TypeContextPolicy(...)})
runtime = PRARuntime.from_pretrained(
    model_id, runtime_config=config,
    agent_config=agent, context_policy=context)
session = runtime.open_session(...)
palette = runtime.activate_capability_candidates(record_ids)
runtime.activate_capability(selected_id)
result = runtime.execute_tool_and_record(..., session=session)
audit = runtime.result_native_index_audit(
    session, result.record.record_id)
if audit.native_index_state == SKIPPED_SIZE_LIMIT:
    detail = runtime.encode_result_region_native(
        session, result.record.record_id, {"rows": [20, 30]})
trace = runtime.inspect()
```

The same facade still accepts direct references, an `ExternalMemoryManager`, a custom discovery engine, and a safe executor. The executed notebook demonstrates model memory, physical materialization, lazy capabilities, compact results, cursors, size-gated native result routing, and selected-region native promotion with an offline Hugging Face model and pure in-memory tools.

4 Physical Materialization

4.1 Planning before copying

Routing produces logical half-open intervals $I_i = (u_i, \ell_i, a_i, b_i)$, naming URI u_i, layer ℓ_i, and token range $[a_i, b_i)$. The planner groups intervals by URI and layer, sorts them, merges overlaps, and then applies a global token budget. If two requested intervals overlap, their common tokens are copied once. The trace reports requested, unique, and dropped token counts, making deduplication and budget truncation auditable.

The portable materializer slices source tensors, transfers selected slices when needed, and packs them by consumption layer. It reports output bytes, transfer bytes, and temporary bytes. It does not infer peak HBM from these values; CUDA profiling records peak allocator state separately.

4.2 Native-reference semantic gate

Reference K/V is captured at source-relative positions. The direct query must therefore begin strictly after every source key that it may attend to. For independent records r with source lengths L_r, the runtime uses

$$p_{q,j} = \max_r L_r + j, \tag{7}$$

not j and not $\sum_r L_r + j$. The maximum avoids query/source position collisions while preserving each record's independent coordinate system. Earlier code started $p_{q,0} = 0$, which invalidated full-reference RoPE parity. This was a runtime bug, not a routing or training deficiency.

The corrected path enters selected native K/V and local K/V into the same family-native softmax. It retains grouped-query K/V heads physically, preserves the model's scaling and causal mask, and keeps the immutable materialization plan active through prefill and every decode call. Compact telemetry records query width, local-cache width, and active native width without retaining attention tensors.

Table 1: HF native-reference correctness ladder. E0 is an offline FP32 mechanism test; E2–E3 reuse the 15-case frozen Qwen3-0.6B Paper 8 ablation.

Gate	Condition	Accuracy	Native tokens	Result
E0	One complete reference; visible/native logits	—	8	5.97×10^{-8} max error
E1	Multiple complete records; boundary/dedup audit	—	144	pass
E2	Selected complete task records; all 28 layers	60.0%	80.2	functional
E3	Paper-3 radius-zero evidence; all 28 layers	0.0%	35.7	insufficient

The bug fix closes the semantic blocker but does not make every sparse geometry adequate. Visible full task scope reaches 73.3%, corrected selected full-record native consumption reaches 60.0%, and sparse late-layer or radius-zero paths remain at 0% on this frozen checkpoint. The first expected token is often ranked correctly even when later generation fails. The remaining boundary is autoregressive composition across materialization depth and layer coverage; training is not proposed as a remedy for a broken native path.

4.3 Cross-model semantic reference gate

We next apply one architecture-neutral protocol to three frozen pretrained models. The exact checkpoints are Qwen/Qwen3-0.6B revision `c1899de2`, `unsloth/Llama-3.2-1B` revision `9535bd9b`, and `google/gemma-3-1b-it` revision `dcc83ea8`. The official Meta Llama revision `4e20de36` remained license-gated to the available account; the experiment therefore uses a public weight mirror and records that substitution rather than implying that the official repository was tested. All runs use eager FP16 CUDA on the GTX 950M, one model per process.

Before the semantic cases, each model must preserve logits, hidden states, greedy generation, and the complete dynamic cache exactly with PRA disabled. Reference registration must retain native K/V-head count and head width without query-head expansion, preserve source-relative positions, and remain active for two explicit decode calls. The four measured conditions are a visible prefix, the same complete reference as native K/V, an explicitly selected complete record, and a family-declared sparse layer profile. Three one-token continuations test a capital, color, and numeric code. This small diagnostic localizes integration failures; it is not an answer-quality benchmark.

Table 2: Cross-model HF semantic gate. Selected-full and sparse columns report one-token exact accuracy over three cases. Full-native errors are against the same visible prefix. KV-head fidelity and request-lifetime checks pass for all rows.

Model	Disabled	Full native	Selected full	Sparse profile	KV heads
Qwen3-0.6B	exact	0.0364 max	66.7%	8/28 layers; 66.7%	8/16 preserved
Llama-3.2-1B mirror	exact	0.0234 max	66.7%	8/16 layers; 33.3%	8/32 preserved
Gemma-3-1B	exact	partial; 9.5410 max	100.0%	2/4 global; 100.0%	1/4 preserved

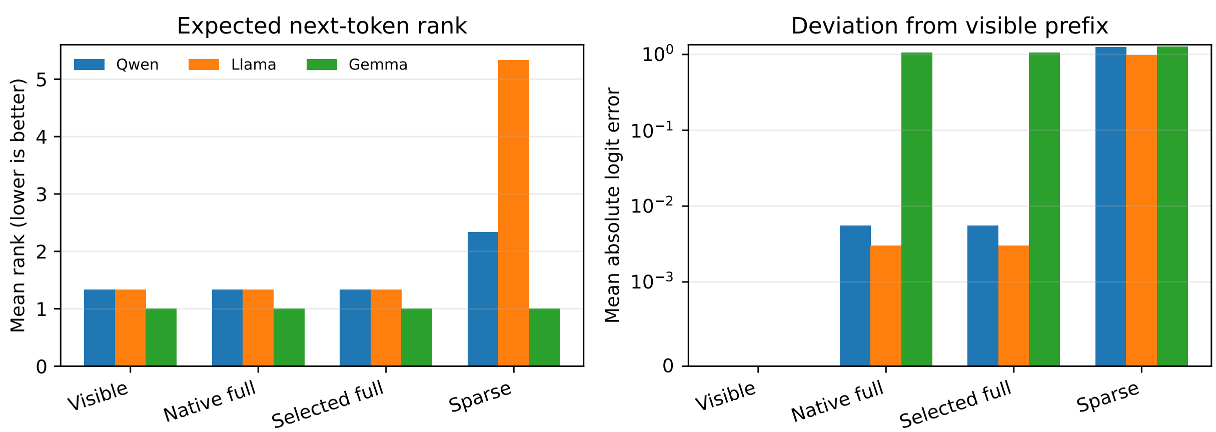

Figure 2: The full and selected-full conditions preserve expected-token rank for Qwen and Llama. Reducing consumer-layer coverage changes semantics despite selecting the same complete record. Gemma's global-only path retains the top token in these cases but has large logit deviation because unchanged local sliding layers do not consume external K/V.

For Qwen and Llama, full-native replay preserves the visible-prefix top token in all three cases. Maximum FP16 logit error is 0.0364 and 0.0234, respectively. The sparse condition exposes the three independent materialization axes: record selection remains fixed, token width remains the complete selected record, but consumer layers fall from 28 to 8 for Qwen and from 16 to 8 for Llama. Mean expected-token rank consequently moves from 1.33 to 2.33 and 5.33, and Llama accuracy falls from 66.7% to 33.3%. Sparse routing therefore does not establish a semantically adequate consumption profile.

Gemma is a useful failed abstraction boundary. Its 26-layer decoder contains four global-attention layers; the adapter intentionally leaves 22 local sliding-window layers native. Disabled parity, single-head MQA storage, positions, and decode lifetime pass, but global-only native replay differs from the visible prefix by 9.5410 maximum and 1.0667 mean absolute logit error. The one-token top answers remain unchanged, but that coarse agreement is not full-prefix equivalence. We therefore report *partial topology*, not cross-family semantic parity.

4.4 Corrected layer-profile calibration

The earlier sparse column was a diagnostic, not a calibrated deployment profile. We therefore inject all eligible layers once and replay one fixed, complete reference identity through a common profile sweep. Qwen uses all 28 decoder layers; Llama uses all 16; Gemma normalizes over its four eligible global-attention layers. Candidate profiles are all layers; contiguous late suffixes of 1, 4, 8, 12, 14, 16, 20, and 24 where defined; early and middle blocks; and evenly spaced blocks. Reference width, query offset, native post-position K/V, GQA/MQA head count, and shared-softmax attention remain fixed. The 102 rows comprise three frozen one-token cases per model. They calibrate SDK behavior but do not replace Paper 3's workload-scale placement study.

Table 3: Corrected fixed-identity layer calibration. ΔLP is target-token log probability relative to the visible-prefix control. Rank is averaged over three cases. "All" means all eligible layers, not necessarily every physical decoder layer for Gemma.

Model	Profile	Consumers	Active layer-tokens	ΔLP	Rank
Qwen3-0.6B	all	28	196.0	+.001	1.33
	last 24	24	168.0	+.389	1.00
	last 8	8	56.0	−.650	2.33
	last 1	1	7.0	−1.705	20.33
Llama-3.2-1B	all	16	106.7	−.002	1.33
	last 8	8	53.3	−1.986	5.33
	last 1	1	6.7	−2.708	16.67
Gemma-3-1B	all four global	4	28.0	−.038	1.00
	last global	1	7.0	−7.818	72.33

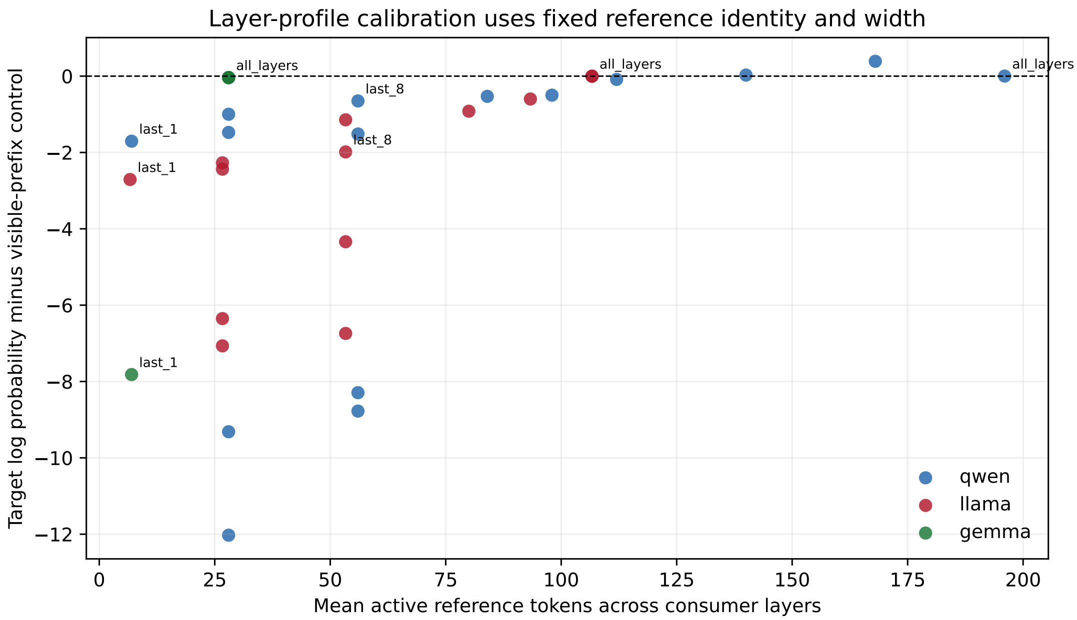

Figure 3: Target-token quality against active layer-token cost with reference identity and token width fixed. The sharp family-dependent trajectories rule out a universal "last eight" product default.

All-layer Qwen and Llama replay are the only tested profiles with maximum full-logit error below .1. Qwen last-24 has the best mean target log probability on these three cases, but its maximum full-logit deviation is 7.14; it is therefore registered under the internal `quality_max` objective but published as `QUALITY_MAX_CANDIDATE`, not as a production-grade `QUALITY_MAX` profile or the balanced product default. Llama requires all 16 layers for the same fidelity criterion. Gemma requires all four eligible global layers to preserve the tested target tokens, yet still cannot reproduce the hidden computation of its 22 unchanged local layers. Thus `balanced` and `reference_correctness` resolve to all eligible layers for all three families in this registry version. The economy profile is an explicit last-one stress point and carries no quality guarantee.

The storage consequence is intentionally separate. Routing-only address state is retained at one late layer, while detail K/V is retained at the resolved consumer layers. On these short references, Qwen detail storage falls from 802,816 bytes for all 28 layers to 688,128 for last 24 and 229,376 for last 8; the routing index remains 2,048 bytes. A service that needs rapid profile switching may ingest the union of candidate detail layers. The production default instead stores only the chosen profile and re-encodes explicitly if a later request asks for missing detail.

4.5 Larger-model profile scaling on Apple Silicon

The small semantic cases above establish a profile contract, not a scaling law. We therefore apply the same normalized suffix space to pinned 4-bit Qwen3-8B, 14B, and 32B checkpoints and Qwen3-30B-A3B on an M4 Pro with 48 GiB unified memory. Each model sees the same five unique annotated-evidence examples per dataset from QASPER, HotpotQA, and 2Wiki. Selection identity and the 384-token source cap are fixed across conditions. This is workload-backed calibration evidence; it is not a routed end-task confirmation because the dataset annotations provide the selected documents.

Table 4: Large-model MLX profile calibration. Concat and segmented agreement are greedy-sequence agreement with selected-text E0. PRA MiB counts active all-layer selected K/V. Every reduced-layer profile remains CALIBRATION PENDING.

Model	Layers	E0 F1	Concat agr.	Seg. agr.	$\lvert\Delta\rvert$LP	PRA MiB	Seg./E0
Qwen3-8B	36	0.236	1.000	1.000	0.189	39.6	1.08
Qwen3-14B	40	0.277	1.000	1.000	0.027	43.9	1.05
Qwen3-32B	64	0.231	1.000	0.867	0.039	70.3	1.03
Qwen3-30B-A3B	48	0.256	1.000	0.933	0.146	26.4	1.12

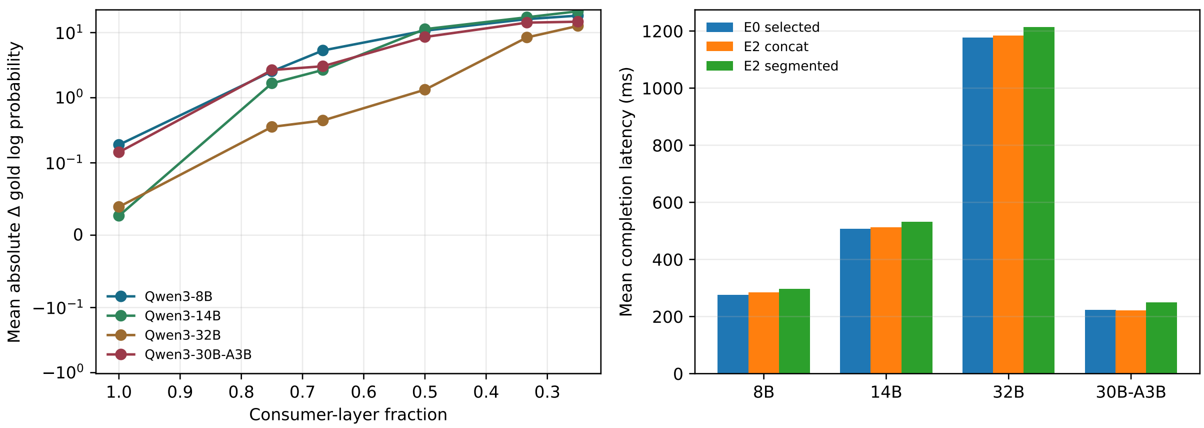

Figure 4: Profile sensitivity and execution time across the dense Qwen ladder and the 30B-A3B control. Larger dense models reduce likelihood sensitivity to missing early consumers, but do not establish sequence-stable sparse profiles.

Full concatenated native K/V preserves all 60 selected-text sequences and is therefore the `REFERENCE_CORRECTNESS` execution path for these rows. The live segmented one-softmax implementation preserves 57/60, with mean absolute gold-log-probability differences of .189, .027, .039, and .146 nats at 8B, 14B, 32B, and 30B-A3B. It remains a kernel candidate rather than a replacement for the exact reference path. The dense last-three-quarter profile becomes less likelihood-sensitive with scale: its mean absolute delta falls from 2.545 to 1.665 to .356 nats. Sequence agreement does not improve monotonically (.267, .467, and .133), so this trend cannot justify a family-level sparse default. The MoE profile uses less active K/V but is more layer-sensitive than dense 32B. Parameter sparsity and semantic-context sparsity are separate profile axes.

These results leave product naming unchanged but strengthen its basis. `REFERENCE_CORRECTNESS` remains all eligible layers. A reduced `QUALITY_MAX_CANDIDATE` may be emitted only with model, revision, workload, materialization, and evidence tier attached. `QUALITY_MAX` remains unassigned; neither a favorable likelihood mean nor a large-model label can promote it without held-out sequence/task validation.

Long-context pressure receipt. The same frozen selections were then held at at most 384 source tokens while same-dataset distractors expanded the full condition to 8K. Selected segmented E2 preserves every selected-text sequence at the measured 8B and 14B points, but the sign of full-minus-selected answer likelihood changes from +1.192 nats at 8B to −1.485 at 14B. This confirms that a profile cannot be promoted from a monotonic larger-model context-tolerance assumption. The physical profile is stable: selected all-layer K/V is 39.6 MiB at 8B and 43.9 MiB at 14B, versus 1,152 and 1,280 MiB for the 8K dense source. At 32B it is 70.3 MiB versus 2,048 MiB; selected segmented E2 preserves 13/15 sequences with a +0.030-nat mean likelihood shift. At the three-case 8B 32K boundary, selected K/V remains 39.6 MiB while dense native K/V reaches 4,608 MiB. These are CALIBRATION rows. The 64K request exceeds the pinned 40,960-position limit and is recorded as unsupported, not silently shortened.

Corrected M5 and profile-policy receipt. A second-host M5 run adds one graph warm-up per path and matched warm/warm and cold/cold tails. Over 30 natural-QA examples per checkpoint, concat E2 preserves every E0 output. Its warm ratios are 1.035, 1.015, and 0.980 for Qwen3-4B, 8B, and 14B; cold usable-context ratios are 0.973, 0.968, and 0.946. The 14B warm point estimate is below E0, but its 95% paired-bootstrap interval [0.922, 1.012] includes parity. Unfused segmented ratios contract from 1.290 to 1.156 and 1.031. The 32B preflight is not forced through swap: 22.86 GB is required against 17.18 GB physical memory, so its row is `NOT_RUN_CAPACITY_GATE`.

A held-out noncontiguous Qwen3-4B calibration also fails to remove any layer group under a 0.05-nat likelihood and 0.90 first-token constraint. The calibrated set remains 36/36 layers and all 12 held-out outputs match E0. This is a negative sparsity result, not evidence for a learned reduced profile. Accordingly, `BALANCED` means all eligible consumer layers; every reduced or segmented realization remains `CALIBRATION_PENDING`.

Finally, corrected Qwen3-8B occupied-context rows keep selected detail at 39.6 MiB and selected-E2 completion at 387.4 ms across 2K, 8K, and 32K source occupancy, with 18/18 selected-text sequence agreement. The full visible path grows from 6.37 to 31.85 GiB peak allocation. These receipts are imported into the product matrix with model revision, host, seed count, and measurement status; they do not upgrade unfused segmented execution to a production kernel.

4.6 Product profile and benchmark matrix

The registry also emits a strict machine-readable product row rather than mixing numbers and labels such as "not measured." Schema 2.0 keeps numeric values, measurement status, and provenance in separate fields for every quality, latency, memory, transfer, scheduler, and reuse metric. Missing values are null with explicit `NOT_MEASURED` or `NOT_APPLICABLE` status; they are never encoded as zero. Unknown columns, duplicate row identities, unsupported integration levels, throughput without a paired quality measure, and invalid status values fail validation. Table 5 is generated from the same JSON artifact used by the SDK.

Table 5: Compact product matrix. Sparse quality-max settings remain candidates until held-out workload calibration closes their evidence tier.

Model	Profile	Level	Quality Δ	Visible ↓	Active KV ↓	TTFT	Status
Qwen	REFERENCE_CORRECTNESS	E2	0.0%	–	0.0%	–	MEASURED
Qwen	QUALITY_MAX_CANDIDATE	E2	4.3%	–	14.3%	–	CALIBRATION PENDING
Qwen	BALANCED	E2	0.0%	–	0.0%	–	MEASURED
Qwen	ECONOMY	E2	-8.8%	–	96.4%	–	MEASURED
Llama	REFERENCE_CORRECTNESS	E2	0.0%	–	0.0%	–	MEASURED
Llama	QUALITY_MAX_CANDIDATE	E2	0.0%	–	0.0%	–	CALIBRATION PENDING
Llama	BALANCED	E2	0.0%	–	0.0%	–	MEASURED
Llama	ECONOMY	E2	-31.1%	–	93.8%	–	MEASURED
Gemma	REFERENCE_CORRECTNESS	E2	0.0%	–	0.0%	–	NOT MEASURED
Gemma	QUALITY_MAX_CANDIDATE	E2	0.0%	–	0.0%	–	CALIBRATION PENDING
Gemma	BALANCED	E2	0.0%	–	0.0%	–	NOT MEASURED
Gemma	ECONOMY	E2	-87.4%	–	75.0%	–	NOT MEASURED

The calibrated layer candidates become useful to an SDK only after their semantics and evidence are inspectable. The current registry publishes four profiles: `REFERENCE_CORRECTNESS`, `QUALITY_MAX_CANDIDATE`, `BALANCED`, and `ECONOMY`. The accepted `QUALITY_MAX` name is deliberately reserved and has no evidence row until workload-scale validation supports it. A semantic row is keyed by model ID, immutable revision, workload, and profile. Engine, engine version, hardware, and dtype identify a physical realization of that row; they do not redefine the profile. This separation prevents one GPU measurement from becoming an implied portable latency property.

Table 6 is generated directly from the packaged JSON registry. Quality is mean target-token probability over the same three fixed continuation cases used above. Retention and delta are recomputed against the model's reference-correctness row. Materialized tokens count selected source tokens, whereas active K/V counts those tokens across consuming layers. Detail savings measure persistent native K/V, so they remain distinct from active attention savings. The compact manuscript table shows active K/V; materialized tokens, detail storage, and the complete physical realization remain in the generated registry and technical table.

Table 6: Model-specific product profiles. This is Smoke evidence over three fixed cases, not workload-scale quality. Quality-max candidates are explicitly Calibration Pending; Gemma rows also remain partial-topology at the mechanism level.

Model	Product profile	Quality	Ret.	Δ	Active K/V	Save	Evidence	Status
Qwen3-0.6B	Ref. correctness	0.4364	100.0%	+0.0000	196.0	0.0%	Smoke	Measured
Qwen3-0.6B	Quality max candidate	0.4790	109.8%	+0.0427	168.0	14.3%	Smoke	Calibration Pending
Qwen3-0.6B	Balanced	0.4364	100.0%	+0.0000	196.0	0.0%	Smoke	Measured
Qwen3-0.6B	Economy	0.3488	79.9%	-0.0875	7.0	96.4%	Smoke	Measured
Llama-3.2-1B	Ref. correctness	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Measured
Llama-3.2-1B	Quality max candidate	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Calibration Pending
Llama-3.2-1B	Balanced	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Measured
Llama-3.2-1B	Economy	0.1972	38.8%	-0.3111	6.7	93.8%	Smoke	Measured
Gemma-3-1B	Ref. correctness	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Partial Topology
Gemma-3-1B	Quality max candidate	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Calibration Pending
Gemma-3-1B	Balanced	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Partial Topology
Gemma-3-1B	Economy	0.0009	0.1%	-0.8744	7.0	75.0%	Smoke	Partial Topology

Qwen's last-24 candidate has 109.8% mean-probability retention while reducing active and detail K/V by 14.3%; this narrow positive result motivates its quality-max-candidate label but does not establish general superiority or a production `QUALITY_MAX` profile. Its registry row pairs `evidence_tier=SMOKE` with `profile_status=CALIBRATION_PENDING`. Qwen economy retains 79.9% on the smoke cases with 96.4% K/V savings. The corresponding economy points are poor for Llama (38.8% retention) and unusable for Gemma (0.1%), which is precisely why the SDK does not ship one universal sparse layer rule. Balanced remains all eligible layers for this registry version.

The semantic label resolves to inspectable mechanism settings. Explicit request fields are applied afterward and therefore take precedence over profile defaults. The runtime trace records `profile_requested`, the resolved profile, its source, registry version, evidence tier, measurement status, and separate product-readiness `profile_status`. A request for the reserved `QUALITY_MAX` label or an uncalibrated model/workload pair returns `CALIBRATION_PENDING` rather than borrowing candidate evidence.

Table 7: Resolved technical realization. Counts refer to eligible physical layers. Compression and search are fixed controls in this semantic smoke.

Model	Profile	Compression	Search	Materialization	Address	Detail	Route	Consumer
Qwen3-0.6B	REFERENCE_CORRECTNESS	none_semantic_smoke	fixed_full_reference	full_selected_record	1	28	1	28
Qwen3-0.6B	QUALITY_MAX_CANDIDATE	none_semantic_smoke	fixed_full_reference	full_selected_record	1	24	1	24
Qwen3-0.6B	BALANCED	none_semantic_smoke	fixed_full_reference	full_selected_record	1	28	1	28
Qwen3-0.6B	ECONOMY	none_semantic_smoke	fixed_full_reference	full_selected_record	1	1	1	1
Llama-3.2-1B	REFERENCE_CORRECTNESS	none_semantic_smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	QUALITY_MAX_CANDIDATE	none_semantic_smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	BALANCED	none_semantic_smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	ECONOMY	none_semantic_smoke	fixed_full_reference	full_selected_record	1	1	1	1
Gemma-3-1B	REFERENCE_CORRECTNESS	none_semantic_smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	QUALITY_MAX_CANDIDATE	none_semantic_smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	BALANCED	none_semantic_smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	ECONOMY	none_semantic_smoke	fixed_full_reference	full_selected_record	1	1	1	1

Runtime fields use an explicit missingness contract. The registry contains TTFT, inter-token latency, tokens/s, p50/p95/p99, throughput, peak HBM and RAM, H2D bytes and time, cache hit rate, index build time, prefetch overlap, and concurrency. None was measured by the layer-quality calibration, so the generated view says NOT MEASURED; it does not reuse the separate microbenchmark later in this paper.

Table 8: Physical realization fields for balanced semantic rows. Missing serving measurements remain explicit.

Model	Engine	Hardware	dtype	TTFT	tok/s	p95	Runtime status
Qwen3-0.6B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED
Llama-3.2-1B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED
Gemma-3-1B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED

4.7 Inherited typed-record product policy

The runtime also forward-ports Paper 7's finalized typed-record policy without reinterpreting its metrics as layer-profile evidence. Per-record compression accepts target-token, maximum-token, and ratio bounds. Typed postings, indexed BM25, and a small embedding index are built independently of the full-body native Q/K size gate. Above that gate, cheap hybrid discovery returns an exact selector; only the selected region is eligible for lazy native encoding and detail-K/V materialization.

In Paper 7's controlled protocol, type-aware compression reduced mean visible tokens from 180.8 to 54.1, a compact ratio of 25%. Automatic recovery was 71.9%, and cheap-hybrid recall at four was 87.5% over 24 cases. These are inherited measurements with their original cohort and provenance, not new Paper 4.5 quality results. Their role here is to define canonical runtime behavior: full native indexing is budgeted, while cheaper address modes and exact backing identities remain available above the gate.

The task smoke invokes the inherited Paper 8 task graph, provenance attachment, and task-structural selector rather than copying expected IDs into the report. It selects the expected record in all 18 model/condition rows (three tasks, three models, visible and selected-full conditions). Selected-full one-token accuracy equals visible-prefix accuracy for every model. Tool and skill disclosure, safe execution, compact results, and session persistence are shared control-plane contracts and pass the executed SDK tests and notebook; we do not label them model-generated tool-use results. The base Llama mirror has no chat template, so no Llama tool-call claim is made.

4.8 Caching and reload amplification

The runtime includes a thread-safe, byte-bounded LRU for opaque warm or hot payloads. Besides hits, misses, and evictions, it records loaded and reused bytes. We define reload amplification as

$$A_{\text{reload}} = \frac{B_{\text{loaded}}}{\max(B_{\text{resident}}, 1)}. \tag{8}$$

An apparently high hit rate can still be costly if the misses repeatedly load large references. Conversely, byte reuse can be high even when object-level hit rate is modest. Both views are needed. Native reusable keys include tenant, user, session, resource, layer, payload variant, source revision, positional frame, materialization layout, and scope signature. Raw backing can be reused across compatible encodings, but per-layer, rebound, and packed payloads require exact geometry identity. The audit accepts only the identical key and rejects changed source, position, packing, or request scope. Per-tenant byte caps evict within that tenant before the global cap is considered, and scope-local cleanup cannot evict another tenant's entries. A concurrent same-URI fixture confirms tenant isolation under pressure and cancellation cleanup. This is logical/runtime isolation, not hard GPU partitioning.

5 Compilation and Engine Boundaries

5.1 Agent, gateway, and engine are independent

The distributed contract has three authorities. The agent owns user-visible task and session semantics, typed records, provenance, and authorization references. The gateway owns transport sessions, logical resource mapping, protocol translation, and explicit capability negotiation. The inference engine owns tensor layout, physical K/V blocks, device placement, attention, and scheduling. The normal wire carries JSON logical identities rather than raw K/V tensors.

The agent boundary is now explicit in code. `AgentTurnContext` contains the conventional chat trajectory and separate context, tool, skill, and task records. `PRAAgent` performs semantic selection but does not choose a wire representation. `NegotiatedRemoteBackend` probes `/v1/pra/capabilities` once at launch or reconnect and resolves TEXT, PRA-FULL, or PRA-DELTA from advertised features. A reachable endpoint without the capability route is ordinary OpenAI; an unreachable endpoint is an error, not a reason to silently downgrade.

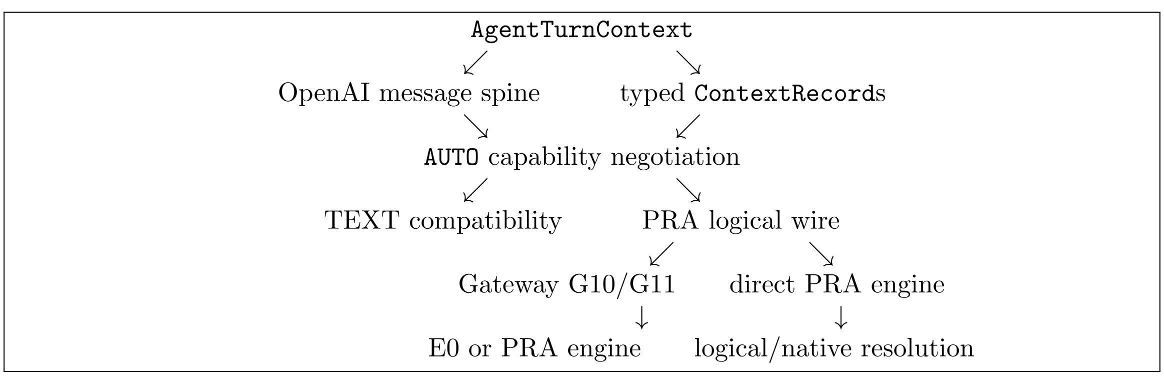

Figure 5: The agent preserves the pretrained conversational spine separately from detached typed context. Compatibility loss occurs only at an explicit TEXT or G10 downgrade; native K/V remains engine-side.

The canonical record projection preserves ID, type, URI, version, source fingerprint, provenance, authorization scope, task ownership/status, available views, selected view, and sharing scope. It never exports Python policy objects, native tensors, or physical page IDs. G10 advertises that its immediate gateway accepts typed records even when its downstream engine is E0; consequently AUTO sends logical records to the gateway and the gateway owns deterministic text injection. G11 forwards record, task, and session semantics. An ordinary server receives no `pra` field.

The implemented `PRAWireRequest` preserves messages, tenant/session/task IDs, resources, facets, budgets, execution-policy hints, provenance, and required capabilities in a version-1 extension of the OpenAI request. `PRAEngineAdapter` reports static capabilities, prepares and closes sessions, and offers generation plus streaming. Four deterministic gateway modes cover both sides of the migration: G00 passes ordinary traffic to an ordinary engine; G10 converts explicitly selected resources to labeled text for a PRA-unaware engine; G01 infers typed resources from system and tool-result boundaries before calling a PRA-aware engine; and G11 preserves an explicit PRA request for a PRA-aware engine. G10 is control-plane PRA with text materialization, not native PRA execution.

Capability negotiation is fail-closed. A request that requires native K/V is rejected by an E0 engine unless it explicitly allows text fallback and selects G10. The trace then names the unsupported capabilities, downgrade, selected logical IDs, gateway mode, correlation ID, and whether native K/V was used. Cross-tenant resources and credential-like request metadata are rejected before translation. Relevance still does not grant source or tool authorization. G01 promotes only explicit record boundaries—system records, tool results, attachments, and already typed messages. It does not infer semantic records from arbitrary unstructured user traffic.

We freeze four integration levels. E0 receives ordinary selected text. E1 understands logical PRA identity, versions, intervals, authorization, session deltas, and fallback semantics, but still realizes context through an ordinary model path. E2 consumes selected resources directly as detached native K/V or an equivalent non-prefix representation with matched geometry, one attention normalization, request-local lifetime, and isolation. E3 adds scheduler-owned placement, prefetch, promotion, eviction, affinity, batching, and distributed movement on top of E2. The local HF adapter is the E2 semantic reference. The OpenAI-compatible HTTP adapter and standalone `pra gateway serve` process implement E0. FreeToken remains reachable through that E0 surface [2]. Companion Papers 6.1 and 6.2 exercise native SGLang-MLX and MLX-LM E2 mechanisms; their provider declarations are available here, but their engine measurements do not promote Paper 4.5 to E3. Paper 6 now exercises native vLLM-Metal V1 generation and scheduler-disjoint pages under a pinned provider, while its generic provider remains E0 unless that bridge is explicitly installed. The portable HF adapter implements E2 native execution and streaming but remains stateless between gateway calls. The configurable remote E1 envelope carries incremental messages and resource operations when the selected engine advertises those capabilities. The HTTP gateway emits OpenAI-compatible server-sent events. Cancellation is cooperative at token boundaries; request-owned references are released only after the decode worker stops. No backing content is included in trace metadata. The capability object names native K/V, logical references, streaming, selected-interval materialization, request lifetime, host/device residency, tenant isolation, phase selection, and scheduler hints independently. The HF reference advertises phase selection through the constrained cache-correct handoff above, but still rejects scheduler hints. This prevents an E2 adapter from being mistaken for an E3 scheduler merely because both accept the same logical wire request.

5.2 Session-Aware Gateway and Conventional Prefix-K/V Reuse

PRA does not replace ordinary prefix caching. A session may simultaneously retain a conventional sequential-prefix K/V cache and separately address PRA records or native detail. The gateway preserves append-only prefixes for ordinary engines and can send logical deltas to PRA-aware remote engines. The first PRA-DELTA turn sends a full inventory. Later unchanged resources emit body-free UNCHANGED operations; changed identity, content fingerprint, authorization, task ownership, or shareability emits ADD or UPDATE, while retirement emits REMOVE. The gateway retains bodies outside its public trace so G10 can render unchanged resources without requiring retransmission. Endpoint restart, reconnect, protocol mismatch, or explicit refresh clears the agent's acknowledged inventory and forces a full resynchronization before delta mode resumes. Figure 6 makes these state classes explicit. The inference engine physically owns the ordinary prefix cache. PRA logically owns resource identities, versions, addresses, and selection; the physical owner of their indexes or selected native K/V depends on E2–E3 integration. A hit or invalidation on one axis therefore says nothing by itself about the other.

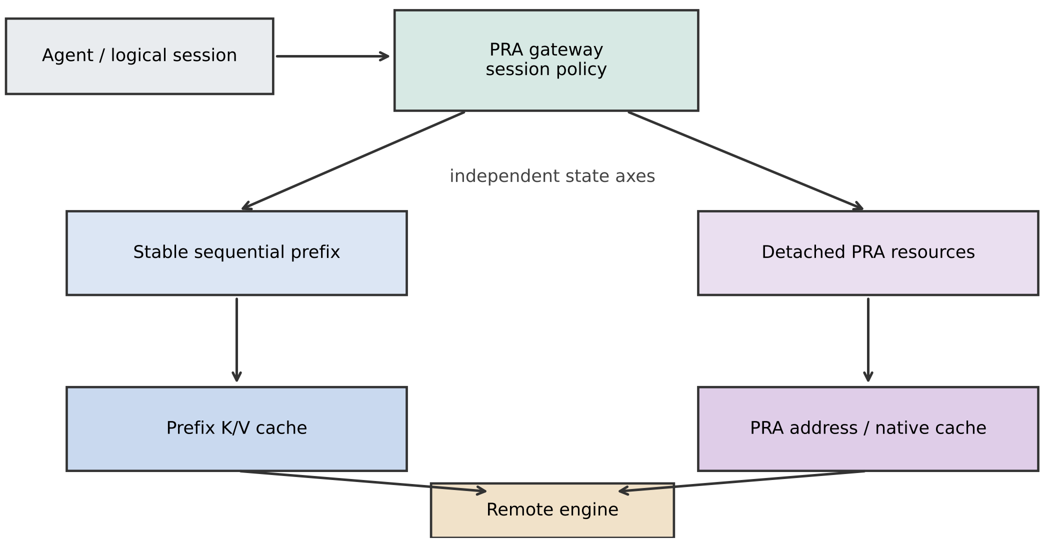

Figure 6: The session-aware gateway maintains two independent state axes. Sequential prompt K/V follows the engine's ordinary prefix-cache policy; detached PRA resources follow the PRA lifecycle and may be text-materialized at E0 or retained natively at E2+.

The wire contract promotes `history_mode`, engine-session identity, prefix handle, cache-affinity key, maximum decode length, and resource operations to typed fields. `FULL` means that the request supplies canonical complete history; `DELTA` means only appended messages; `AUTO` resolves from capabilities rather than message count. Stable resources similarly produce `ADD`, `UPDATE`, `REMOVE`, or `UNCHANGED` operations while preserving ID, URI, version digest, record type, and authorization scope.

A versioned engine registry resolves OpenAI-generic, vLLM, SGLang, FreeToken, Ollama, Hugging Face, llama.cpp, MLX, and custom profiles. Unknown compatible servers safely remain E0, full-history, and cache-telemetry-unknown. vLLM and SGLang profiles represent automatic prefix-cache capability but do not infer native PRA from an engine label. Companion E2 mechanism results are registered through explicit providers and remain separate from Paper 4.5 evidence. The local HF reference remains E2-native but stateless across calls; the session/resource-delta row is exercised by a deterministic remote-engine contract fixture. Explicit configuration may override registry defaults; resolved capabilities, rather than labels, control transport.

For G10, the former message-zero synthetic system block remains only as a legacy experimental control. The default inserts selected evidence into the current user turn, after the byte-identical prior serialized conversation. Role-sequence tests cover Qwen, Llama, Gemma, and generic OpenAI-style transports. This placement preserves template validity and the previous prefix. It does not imply that changing evidence is cacheable: only the prior sequential prefix is reusable.

The sidecar is keyed by tenant, logical session, and model. It retains only ephemeral engine handles, prefix fingerprints, resource versions, policy digests, and affinity hints; durable canonical history continues to use `SessionService`. Preparation is idempotent, explicit close releases engine state, and cancellation of one stream does not close its reusable session. Model revision, chat-template, visible-prefix, history-rewrite, engine-restart, and invalid-handle changes invalidate the relevant handle. Updating one detached resource emits a resource delta without invalidating an otherwise stable conversation prefix. The cache-affinity key is only a future scheduler hint; no worker-placement result is claimed.

5.3 Agent-side compatibility before native engine work

The pre-SGLang deployment gate asks whether PRA's logical record semantics can survive an existing agent without patching its model runtime. The `DeepSeekHarnessPRAAdapter` consumes durable tool-result, session-reference, and attachment events. The `PiCodingAgentPRAAdapter` consumes Pi's public tool-completion and completed-message events. Both deduplicate event identities, retain typed provenance and task/session IDs, and emit the same tensor-free `PRAWireRequest` used by the HTTP gateway.

Across 10 deterministic five-seed contract cases, the DeepSeek and Pi bridges each pass 100.0% and 100.0% of identity, typing, task-metadata, deduplication, and explicit-fallback checks. In every G10 case the ordinary engine receives labeled evidence text, zero logical resources, and no native-K/V claim. These tests validate the integration boundary, not the quality or latency of either upstream agent.

5.4 Compilation ladder

The eager path is the correctness baseline. The first compiled primitive is a paired `index_select` over K and V with dynamic selected-token count. Cold compile time and warm latency are separate rows, and parity is checked against eager outputs. Failure never falls back into an eager row.

Triton, custom CUDA, and fusion follow profiling rather than precede it. The candidate hot paths are interval gather, deduplication/remap, packing, and position preparation. A kernel is justified only if it improves the complete request after launch, transfer, and synchronization costs. This ordering prevents a fast isolated kernel from becoming a misleading end-to-end result.

5.5 Thin vLLM handoff

PagedAttention showed how virtualized K/V blocks improve serving utilization and batching [4]. Deep PRA integration would additionally make logical PRA blocks, semantic residency, and retrieval-aware prefetch visible to the scheduler. That is outside this paper's boundary.

The implemented `VLLMThinBackend` instead emits a `VLLMThinRequest`: prompt, stable selected URIs, materialized token count, and ordinary K/V metadata. Semantic scores do not cross the boundary. An ordinary engine can consume already selected memory without owning retrieval policy. When no executor is installed, the backend produces the handoff for inspection and raises on generation rather than pretending to serve.

6 Relation to Existing Systems

PRA is complementary to several kinds of memory optimization. H2O and SnapKV select or retain influential cache entries inside an active sequence [7, 5]. PRA begins from stable external identities and query-time addressability, so its inactive detail K/V may live outside the accelerator. These methods can eventually share page layouts or eviction mechanisms, but their selection semantics differ.

Prompt Cache and CacheBlend reuse precomputed prompt or retrieved-knowledge state [3, 6]. Their lesson is directly relevant: recomputation and transfer can dominate. PRA adds bounded semantic selection before native K/V consumption. RetrievalAttention similarly joins vector retrieval with long-context attention, reinforcing the need to count index and transfer work rather than only selected attention work.

GQA reduces the number of physical K/V heads [1]. Our runtime preserves this layout through warm storage and materialization. It does not expand K/V to query-head count as an implementation convenience.

7 Experimental Design

7.1 Questions and claim levels

The initial gate asks four narrow questions. First, does the portable selected- K/V implementation preserve exact tensor parity? Second, how large are eager gather and interval-pack costs? Third, how much does CPU residency add for the same selected payload? Fourth, can the local toolchain execute the compiled path? Quality and routing selection remain frozen.

Claims are labeled by evidence. *Measured* means executed on the recorded host. *Inherited* means read from a checked-in earlier experiment with its own protocol. *Contract only* means code exists but no external engine ran. *Architectural* means only an interface comparison is made.

7.2 Gateway session and prefix protocol

The controlled gateway fixture executes five turns with changing selected records under four conditions: legacy message-zero G10 insertion, prefix-preserving G10, E0 message deltas, and an E1 logical message/resource-delta contract. It records whether each prior serialized message sequence remains an exact prefix, the reusable-message fraction, invalidations, JSON message and resource bytes, delta bytes, and session reuse. The workload is deterministic and uses a simulated adapter so that transport semantics can be isolated from model quality and network variance. Byte counts refer to the fixture's JSON envelopes, not tokenizer output or K/V allocation. Physical cache hits remain unknown unless adapter telemetry supplies them.

7.3 Negotiated agent-transport protocol

A second deterministic fixture follows one task through open, tool-result, waiting, resume, and complete turns. Its detached stream contains a long document, an atomic tool schema, a skill record, versioned task state, and a large tool result. The conversational spine grows normally. We freeze the same selected record identities and compare complete TEXT rendering, PRA-FULL, and PRA-DELTA JSON requests. Reported bytes include OpenAI fields and PRA metadata; visible-token estimates count message text only. Selection parity, task-status projection, and provider-native tool-schema preservation are hard assertions. The fixture does not call a language model, so it measures protocol semantics and retransmission rather than answer quality or TTFT.

7.4 Portable primitive protocol

The new benchmark uses PyTorch 2.12.1 with CUDA 12.6 on Windows 10, Python 3.10.11, and an NVIDIA GeForce GTX 950M with compute capability 5.0. Each synthetic tensor has four physical K/V heads, width 64, and 8,192 candidate tokens in float32. Selection admits 512 tokens (6.25%). Batch sizes are one and four. CPU rows use five warmups and 30 timed repetitions; CUDA rows use ten warmups and 50 repetitions. Every CUDA timing synchronizes before and after the operation.

Indexed gather uses evenly distributed selected token IDs. Interval packing uses eight intervals under the same 512-token budget. The hierarchy comparison holds intervals and output bytes fixed while moving sources among HBM, pinned CPU, and ordinary CPU. Compiler errors are artifact rows. The benchmark is a mechanism profile, not TTFT or throughput for a language model.

The layout sweep holds 8,192 total source tokens across four references and two layers. It packs 32-token pages in layer-major, reference-major, chunk-major, or block-major order, then restores an identical 512-token logical selection. Store construction is a cold cost; reported layout gather latency follows the same warmup and repetition protocol as the other device rows.

7.5 Inherited profiles

Two earlier artifacts answer adjacent questions. The Paper 2 public-API demo ran frozen Qwen3-0.6B over two QASPER and two HotpotQA examples. It records reference ingest, query encoding, routing, generation, K/V fractions, transfers, and peak GPU allocation. The Paper 4 lifecycle fixture ran eight deterministic resources through authenticated cold, warm, and hot states. Its answer proxy is required-document availability, not language quality. We do not pool these measurements with the new microbenchmark or compute cross-host speedups.

7.6 Execution-policy mechanism profile

We instantiate the same random-weight, three-layer tiny Llama configuration for five seeds and ingest two stable synthetic references. Every condition uses three active PRA layers, deterministic decoding, and at most three generated tokens. We compare request/shared at first and last routing layers, request/per-layer, token/shared at first and last routing layers, and token/per-layer. The profile records routing and selection epochs, temporal and layer Jaccard, selected-identity agreement, and generation wall time. It tests execution semantics and relative routing work, not language quality or production latency. EOS can terminate before three tokens, so token-level operation counts are averaged over the actual forwards.

8 Results

8.1 Stable prefix and session-delta contract

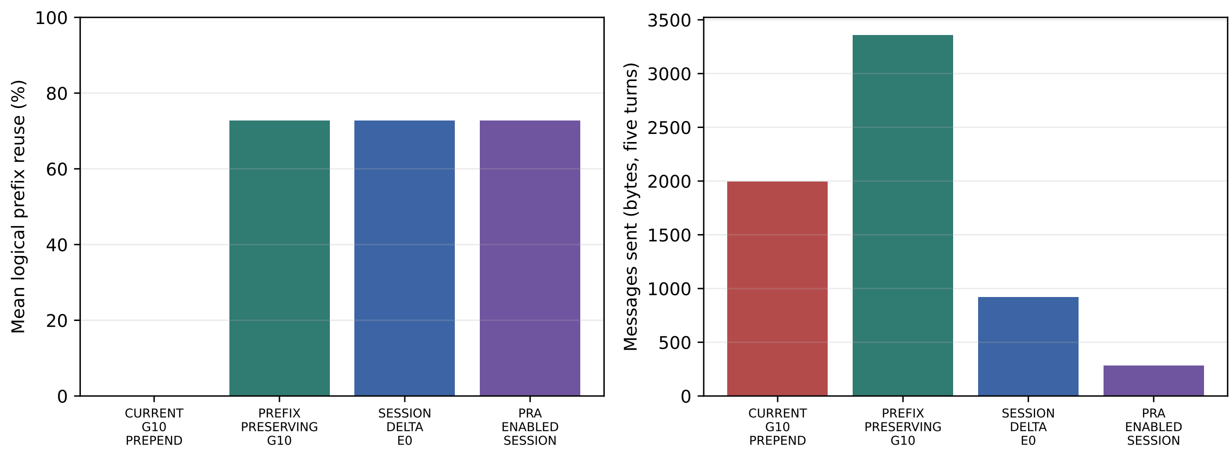

Figure 7: Five-turn deterministic gateway profile. The left panel reports logical reuse of the exact prior serialized message prefix after turn one; the right reports message bytes sent. Resource bytes are reported separately in Table 9.

Table 9: Controlled gateway session profile. Stable turns use the four turns after initial setup. Reuse is logical serialized-message reuse, not an engine-reported cache hit.

Condition	Stable turns	Reuse	Invalidations	Msg bytes	Resource bytes
CURRENT_G10_PREPEND	0/4	0.0%	4	1995	0
PREFIX_PRESERVING_G10	4/4	72.8%	0	3355	0
SESSION_DELTA_E0	4/4	72.8%	0	921	0
PRA_ENABLED_SESSION	4/4	72.8%	0	281	2297

The legacy message-zero G10 control changes the leading evidence block on every eligible turn, preserving zero of four prior prefixes and recording four invalidations. Moving evidence to the current turn preserves all four prior prefixes, with a mean logical reuse fraction of 72.8%. Full-history prefix-preserving G10 sends 3,355 message bytes because it still retransmits history. The session-delta E0 contract retains the same logical prefix while reducing message transport to 921 bytes; four turns reuse the prepared engine session.

The E1 contract sends 281 message bytes and 2,297 resource-envelope bytes, using resource additions, updates, and removals rather than repeating resource contents indiscriminately. Its combined delta bytes are not lower than the tiny text fixture because typed resource metadata and operation envelopes are explicit. This is a semantic and accounting result, not evidence that PRA transport is always smaller. All four conditions report physical prefix-cache hits as unknown. No vLLM/SGLang affinity, TTFT, throughput, or scheduler result is inferred from logical prefix stability.

8.2 Typed transport avoids repeated record bodies

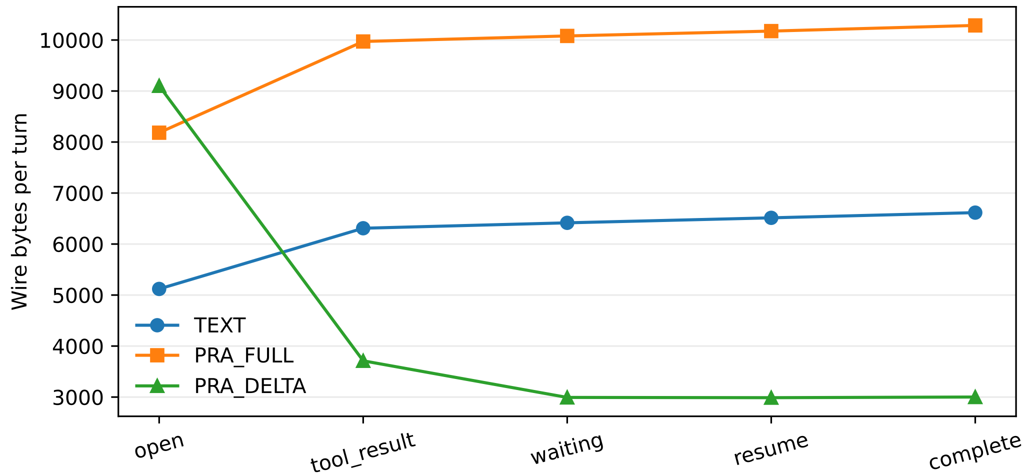

Figure 8: Per-turn JSON bytes for the five-turn task/tool workflow. PRA-FULL preserves explicit metadata but repeats all resource bodies; PRA-DELTA sends a body only for ADD or UPDATE and keeps unchanged resources logical.

Table 10: Negotiated transport contract over one deterministic long-running task. Byte totals are wire serialization, not model tokens or latency.

Mode	Total bytes	Resource bodies sent
TEXT	30,976	24 rendered copies
PRA-FULL	48,695	24
PRA-DELTA	21,795	8

PRA-FULL is larger than TEXT in this fixture because it carries explicit identity, views, provenance, task state, and authorization metadata in addition to complete bodies. PRA-DELTA reduces total wire bytes by 29.6% relative to TEXT and 55.2% relative to PRA-FULL, while reducing body transmissions from 24 to 8. All five turns preserve selected identity, task status, and the provider-native tool schema. These are transport invariants, not evidence that logical PRA improves model quality. A matched model benchmark must hold selection, model, positions, profile, and generation fixed.

We therefore repeated the comparison with a real MLX-LM Qwen3-0.6B model on ten held-out QASPER selections. Each condition used the same chat template, selected records, task state, provider-native tool schema, greedy decoder, and 24-token budget. The gold answer remained in an out-of-band evaluator rather than request metadata. TEXT, PRA-FULL, and PRA-DELTA produced byte-identical model messages and identical outputs in 10/10 examples; their mean gold-answer log probability was also identical at -40.35625. Mean TTFT was 507, 506, and 481 ms, respectively, but ten independent first turns do not support a latency-ordering claim. Their mean wire sizes were 5113, 9475, and 10919 bytes; first-turn typed metadata has a cost before delta reuse begins.

The long-session fixture makes that reuse visible. At 50 turns, including one forced reconnect and full resynchronization, TEXT sends 638,552 bytes, PRA-FULL sends 840,632, and PRA-DELTA sends 162,253. Delta transport is therefore 74.6% smaller than repeated TEXT and 80.7% smaller than PRA-FULL. It sends 21 resource bodies rather than 275 while preserving selection, task state, and tool schema.

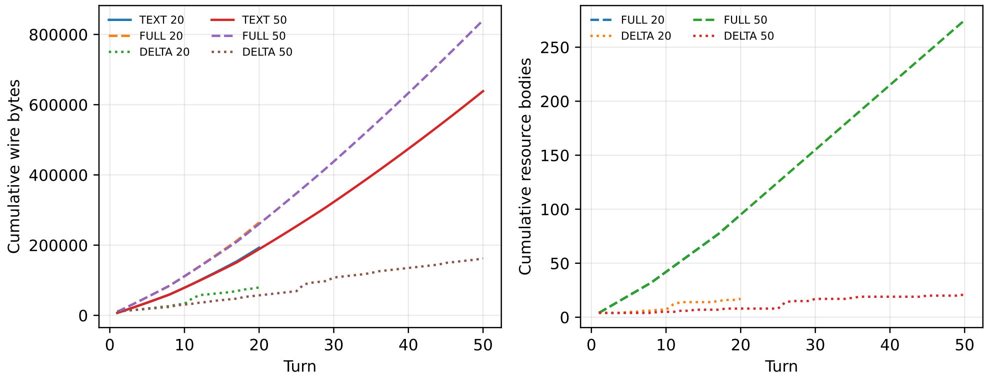

Figure 9: Cumulative wire bytes over 20- and 50-turn workflows. The reconnect forces one resource resynchronization; unchanged resources remain body-free on the other delta turns.

8.3 Selection sharing removes repeated routing work

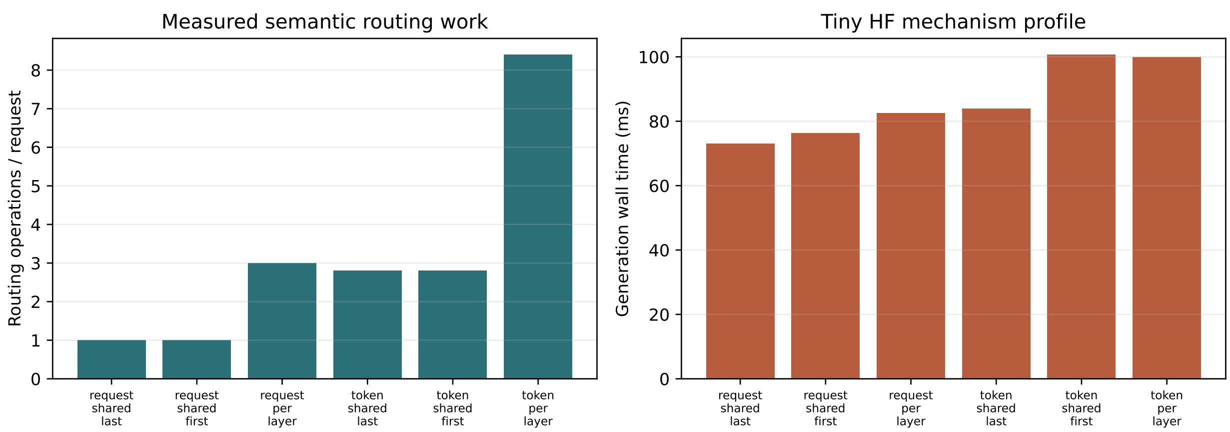

Figure 10: Five-seed tiny-HF mechanism profile. Routing operations validate the execution contract; wall time is host- and model-specific and is not a serving speed claim.

Table 11: Implemented policy points in the five-seed mechanism profile. Times are means over random-weight tiny models and do not measure answer quality.

Condition	Routing operations	Generation ms
Request/shared, last	1.0	73.1
Request/per-layer	3.0	82.6
Token/shared, first	2.8	100.7
Token/per-layer	8.4	100.0

Request/per-layer performs exactly three routing decisions, but amortizes them over generation. Token/shared and token/per-layer observe the same average number of autoregressive forwards; sharing reduces routing work by 3.0×, from 8.4 to 2.8 operations. Tiny-model wall times are effectively tied for the two token modes, which means Python and model overhead dominate at this scale. The routing-layer sweep also changed selected identities, confirming that routing layer is a semantic parameter rather than a scheduling-only knob. The stable selections within these short completions yield temporal Jaccard one; longer trained-model studies are needed before making a churn or quality claim.

8.4 Portable eager primitives

Table 12: Median warm latency in milliseconds. All measured outputs match the eager reference exactly. Each output contains 512 selected tokens per request.

Device	Primitive	Batch 1	Batch 4
CPU	indexed gather	0.133	0.835
CPU	interval pack	0.228	0.938
CUDA	indexed gather	0.253	0.478
CUDA	interval pack	0.415	0.613

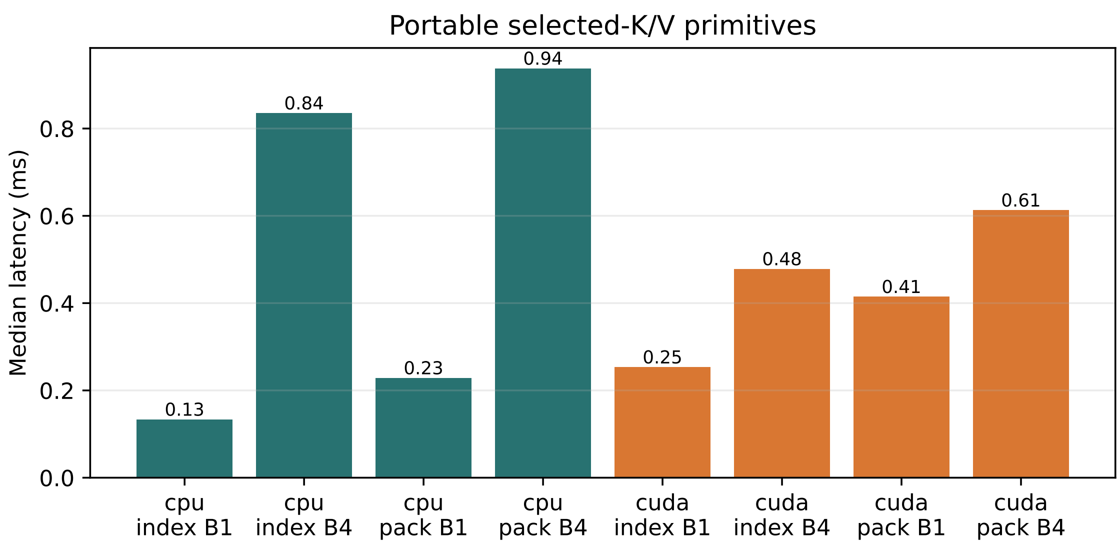

Figure 11: Portable primitive latency. This older GPU does not make small batch-one gathers faster than the CPU, but CUDA scales better at batch four.

The indexed CUDA path is 0.253 ms at batch one. The eight-interval pack is 0.415 ms because it launches and concatenates multiple slices. At batch four, CUDA indexed throughput increases to roughly 4.28 million selected tokens/s, whereas the CPU reaches about 2.45 million million. These values identify packing and fragmentation as optimization targets; they do not include routing or attention.

8.5 Residency dominates the selected payload

Table 13: Median CUDA interval materialization latency for an identical selected payload. Output size is 1 MiB at batch one and 4 MiB at batch four.

Source residency	Batch 1 (ms)	Batch 4 (ms)
HBM resident	0.424	3.371
Pinned CPU → CUDA	1.592	6.655
Ordinary CPU → CUDA	2.157	8.660

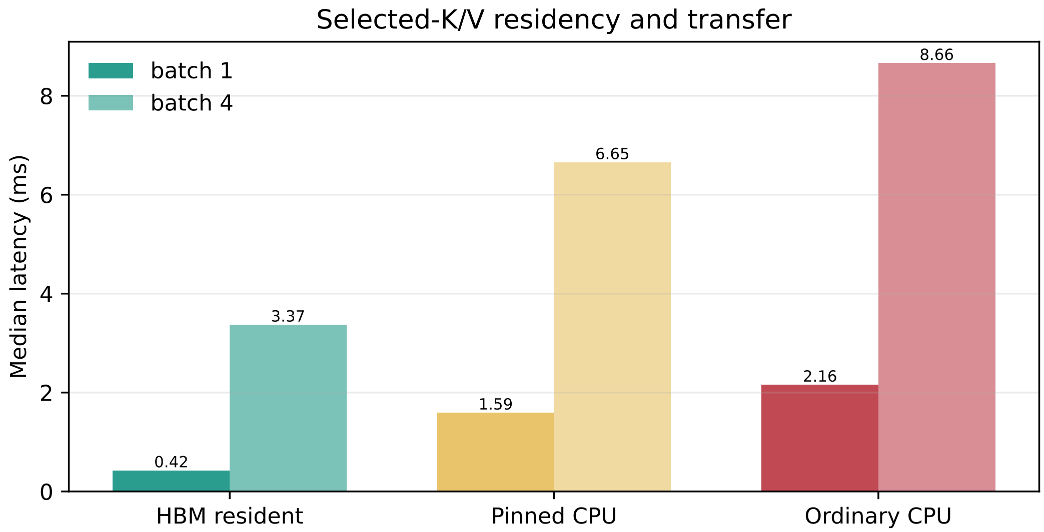

Figure 12: The same logical selection has different physical cost depending on residency. Pinned transfer helps, but does not erase the hierarchy boundary.

At batch one, pinned transfer is 3.75 times the HBM-resident cost, and ordinary CPU transfer is 5.08 times that cost. This is the central result of the initial systems gate. A configuration that moves inactive K/V off accelerator reduces residency, but request latency then depends on locality, cache reuse, and transfer overlap. Any future headline must report both memory and latency.

8.6 Physical layout is secondary to fragmented remapping

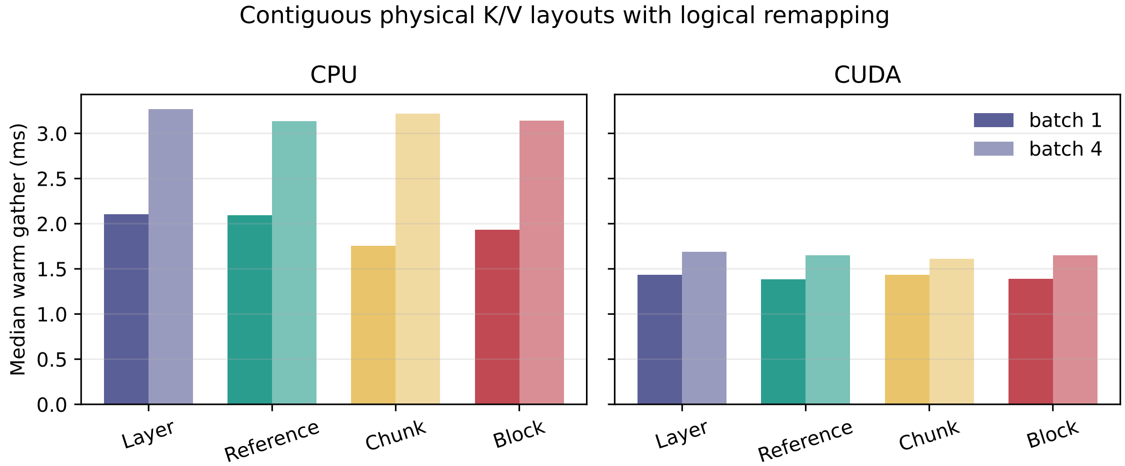

Figure 13: Warm materialization from four physically distinct contiguous stores. Every layout restores the same logical per-layer K/V before attention.

Layer-major, reference-major, chunk-major, and block-major stores place the same four-reference, two-layer native K/V in different contiguous orders. A compact placement index maps URI, layer, and logical page offsets back to physical ranges. All outputs match the unpacked reference exactly. On CUDA, the slowest and fastest medians differ by only 3.8% at batch one and 5.0% at batch four. The nominal fastest layouts are reference-major and chunk-major, respectively, but those small differences do not justify a universal preference on this synthetic cohort.

More importantly, all layout-remap paths take roughly 1.4–1.7 ms on CUDA, several times the single-source interval pack. They reconstruct selected logical ranges across Python placement records and page fragments. The next optimization target is therefore indexed vectorized remapping and fewer launches, not a claim that one physical ordering has won.

8.7 Compilation is a negative capability gate

The `torch.compile` API is present, but neither local compiled path is valid. CPU Inductor cannot find the Microsoft C/C++ compiler. CUDA compilation rejects compute capability 5.0 because the Triton backend requires capability 7.0 or newer. No warm compiled latency is reported and no eager result is relabeled as compiled. Triton and custom CUDA therefore remain untested.

8.8 Earlier profiles locate larger costs

The new primitive measurements explain the cost of moving an already selected payload. Earlier end-to-end artifacts provide the complementary scale: how much time was spent before and after materialization when the public model API and authenticated resource lifecycle were exercised.

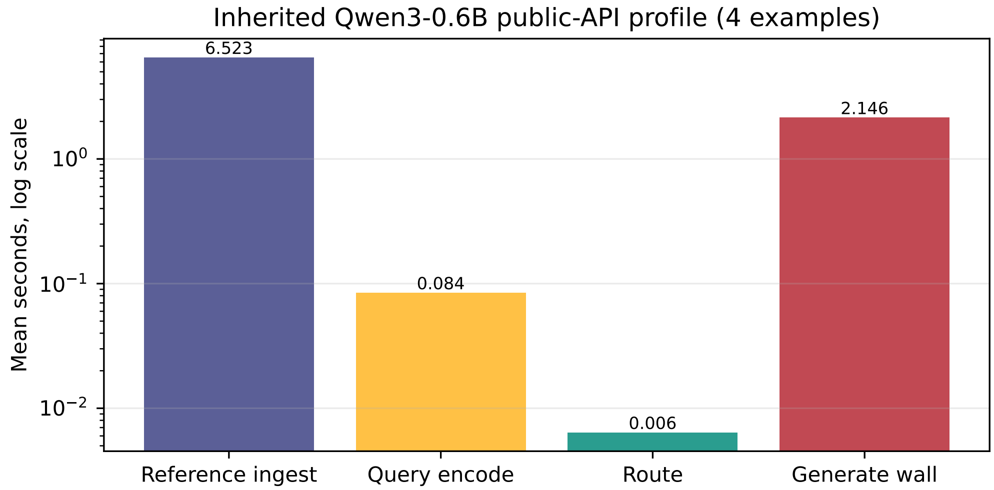

Figure 14: Inherited Paper 2 Qwen3-0.6B profile over four examples. The log scale shows that routing time is small relative to native reference encoding and generation on that protocol.

The inherited profile reports 6.40 ms mean routing time, 6.52 s mean reference ingest, and 2.15 s mean routed generation wall time. Mean evidence recall is 0.667 and materialized detail K/V is 0.066 of the candidate total. This supports optimizing cache reuse and encoding amortization before replacing the router with a custom kernel. The cohort contains only four examples, so it does not establish a general workload distribution.

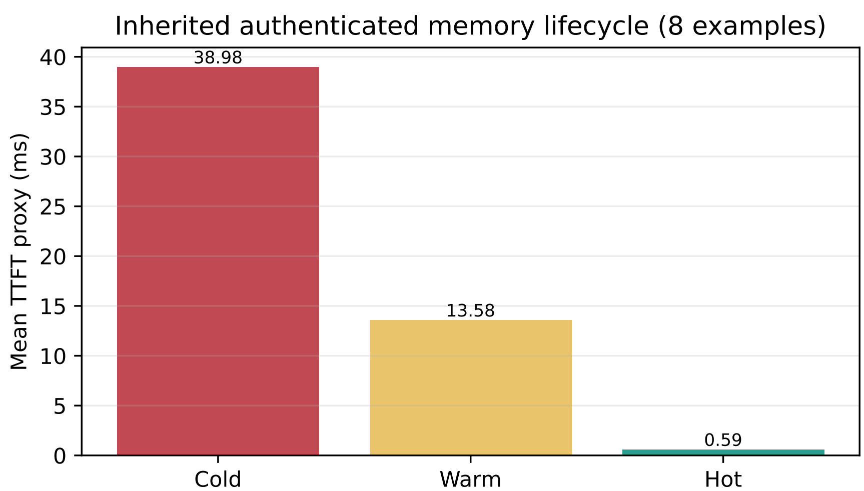

Figure 15: Inherited Paper 4 lifecycle proxy over eight examples. Cold includes fetch and native encoding; warm reuses native encoding; hot reuses selected K/V.

The lifecycle TTFT proxy falls from 38.98 ms cold to 13.58 ms warm and 0.59 ms hot. Required-document recall is 0.875. The result validates state transitions and shows the size of the reuse opportunity, but it is not an LM answer-quality or production-serving measurement.

8.9 Semantic storage lifecycle

A 120-turn mixed trace compares plain LRU, size-aware LRU, and weighted task/type-aware retention at the same 80 KB WARM quota. Weighted retention raises the utility-weighted hit rate from 0.700

to 0.796, preserves every long-running task and shared-document access, and reduces SOURCE reconstructions from 214 to 184. Its estimated context-resolution cost falls from 1,170 to 936 ms. These are policy-trace results; the extra Python scoring work is not presented as engine serving speed.

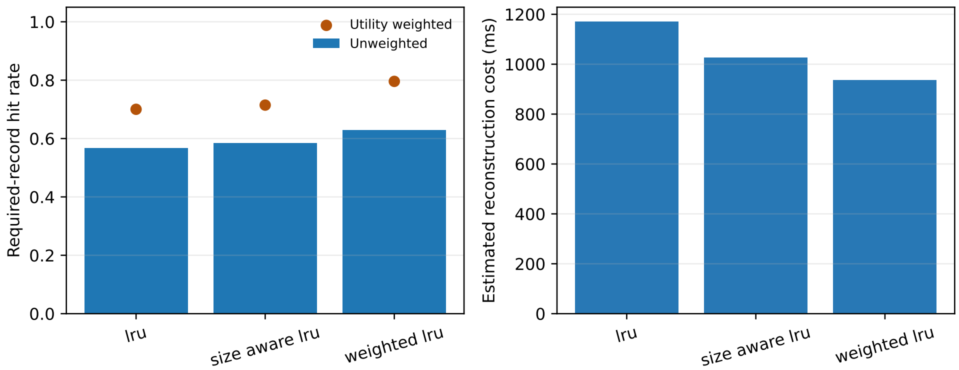

Figure 16: Equal-quota retention on a mixed task, document, and tool-result trace. Weighted policy protects active task state and shared documents while allowing reconstructable transient records to churn.

The runtime now makes the cache hierarchy a shared semantic contract rather than an engine-specific accident. Each typed record has an authoritative SOURCE representation and may have derived native detail in COLD, WARM, or HOT form. HOT means attention-ready and request-pinnable; WARM means lossless native detail suitable for fast promotion; COLD means capacity-oriented persistence with lossless compression and optional, separately named K/V quantization. The physical HOT object remains an engine concern—vLLM pages, an independent SGLang object, or MLX arrays—but record type, task status, dependencies, reconstructability, grace periods, and quotas are decided once by `PRAStorageManager`.

The deterministic policy retains plain LRU, size-aware LRU, reuse-count, and reload-cost controls, and adds weighted LRU. Its score combines recency, frequency, reconstruction cost, typed-record prior, task state, authorized sharing, dependency count, byte cost, and reconstructability. Open, blocked, or waiting tasks enforce minimum WARM lifetimes. Completion schedules delayed compaction rather than deletion; an open dependent task cancels aggressive compaction. Session close releases transient native detail without treating a native-cache quota as permission to delete SOURCE. Newly encoded detail first observes a grace period, avoiding speculative persistence of short-lived tool and terminal records.

Table 14: Shared lifecycle controls over HF, vLLM, SGLang, and MLX labels with five seeds each (20 runs). Rates are policy outcomes under a fixed synthetic pressure trace, not end-task model quality.

Control	Baseline	Semantic lifecycle
Shared document retained under pressure	0	1
Open-task record retained	—	1
Dependent completed-task record retained	—	1
Transient persistence writes	20	0
COLD native-byte ratio	1.00 FP32	0.25 int8

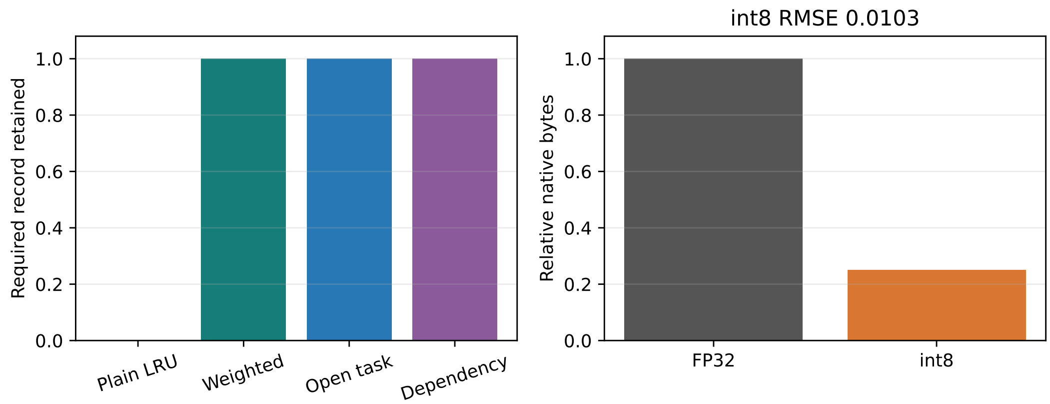

Figure 17: Engine-neutral retention controls and generic symmetric-int8 byte transport. The int8 result is a codec diagnostic; model-specific quality and active-attention bytes remain separate gates.

Lossless HOT→WARM→COLD→HOT recovery and SOURCE reconstruction are exact in 20/20 runs. Under a 24-byte pressure fixture, plain LRU loses the shared document in every run whereas the weighted policy retains it in 20/20; active-task and downstream-dependency protections also hold in 20/20. A persistence grace period reduces writes for the transient closure trace from 20 to zero. Generic symmetric int8 occupies 0.25 of FP32 bytes with 0.0103 RMSE, 0.28 ms median quantization, and 0.07 ms median dequantization on this host. This does not supersede the MLX model-specific int8 quality result, nor show that WARM and COLD improve generation latency. It establishes policy parity, exact lossless recovery, and a measurable reason to defer transient writes; natural lifecycle distributions remain open, while the following small probe begins engine-native promotion measurement.

The next probe replaces the in-memory HOT bridge with live attention objects: reserved vLLM-Metal pages, SGLang MLX arrays backed by the built-in HiCache file API, and MLX arrays. Each of three natural QASPER selections is generated from HOT, demoted to lossless WARM, promoted and generated again, moved through int8 COLD, and finally recovered by a newly constructed lifecycle manager. MLX-format WARM storage is layer segmented: each K and V array has an atomic manifest entry and checksum, and a consumer profile can mmap only requested layers. HOT promotion retains the lower durable copy, so a process failure during an active request does not erase the recoverable representation.

Table 15: Three-example live-engine storage mechanism probe. Ratios include request-time promotion and generation but exclude background demotion and COLD conversion. Prefix is the mean common HOT/int8 output prefix in tokens.

Engine	WARM exact	int8 exact	int8 first	Prefix	WARM/HOT	COLD/HOT
mlx-lm	100%	0%	67%	1.3	1.97	1.84
sglang-mlx	100%	0%	67%	0.7	1.90	1.89
vllm-metal	100%	0%	67%	4.0	1.96	2.00

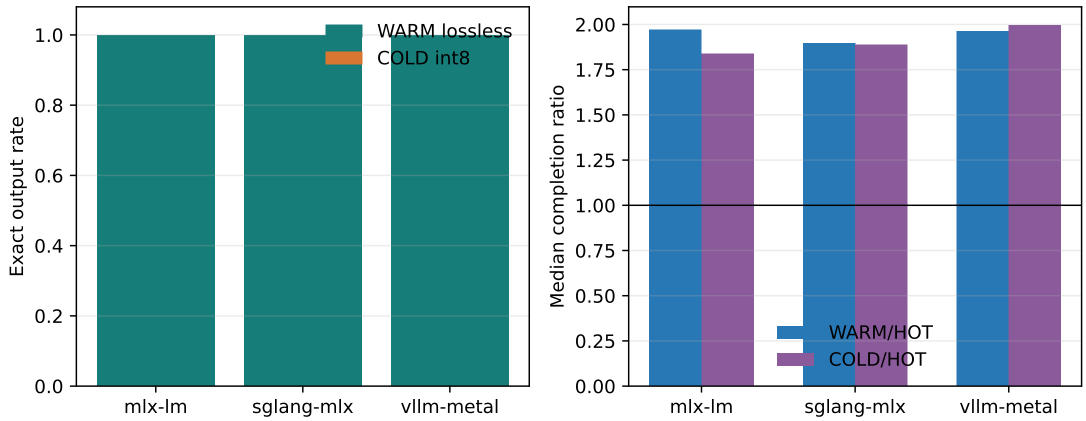

Figure 18: Lossless WARM preserves all three outputs per engine, whereas int8 COLD changes every 24-token sequence. The latency panel measures a tier hit at request time; background WARM and COLD transitions are reported separately.

WARM output is exact in 9/9 engine–example pairs, and restart promotion succeeds for every engine. Median WARM/HOT request ratios are 1.97 for MLX, 1.90 for SGLang, and 1.96 for vLLM; the corresponding COLD/HOT ratios are 1.84, 1.89, and 2.00. Median background WARM transitions cost 348, 158, and 362 ms, while COLD conversion costs 940, 701, and 1,016 ms. These are small mechanism cohorts, not tail-latency estimates. Int8 COLD is 0/9 sequence-exact, with first-token parity in 6/9 and short mean common prefixes. The signed F1 changes are noisy on three low-accuracy examples and do not rescue sequence equivalence. Named product profiles therefore keep lossless COLD as the default; int8 remains explicit, model-calibrated, and disabled unless its own quality gate closes.

We then scale the same live protocol to 20 examples per cohort: QASPER, HotpotQA, and 2WikiMultiHopQA at Qwen3-0.6B, plus QASPER at Qwen3-1.7B, for each engine. Five smaller QASPER cohorts add Llama-3.2-1B to all three engines and Gemma-3-1B to vLLM-Metal and MLX-LM. Lossless WARM reproduces HOT generation in 265/265 pairs and all 17 managers recover after restart. Diagnostic int8 COLD is exact in only 61/265 pairs. The pinned SGLang-MLX build rejects Gemma's mixed global/sliding-window topology before PRA attachment; this is a backend compatibility boundary, not a PRA parity failure. Table 16 keeps model and dataset identities separate; its percentiles describe finite cohorts rather than an online arrival distribution.

Table 16: Expanded live lifecycle matrix. Latencies include request-time promotion and generation; background demotion/conversion remains separate.

Engine	Model	Data	n	WARM exact	int8 exact	HOT p50	WARM p50	WARM p95
mlx-lm	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	40.0%	191	275	281
mlx-lm	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	15.0%	328	658	710
mlx-lm	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	15.0%	329	707	817
mlx-lm	Qwen3-0.6B-4bit	qasper	20	100.0%	15.0%	326	672	714
mlx-lm	Qwen3-1.7B-4bit	qasper	20	100.0%	20.0%	337	723	744
mlx-lm	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	391	480	502
sglang-mlx	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	40.0%	242	287	302
sglang-mlx	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	15.0%	310	525	565
sglang-mlx	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	20.0%	320	543	592
sglang-mlx	Qwen3-0.6B-4bit	qasper	20	100.0%	15.0%	307	538	604
sglang-mlx	Qwen3-1.7B-4bit	qasper	20	100.0%	20.0%	374	614	632
vllm-metal	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	60.0%	299	428	453
vllm-metal	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	30.0%	351	738	803
vllm-metal	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	35.0%	366	737	832
vllm-metal	Qwen3-0.6B-4bit	qasper	20	100.0%	10.0%	360	714	779
vllm-metal	Qwen3-1.7B-4bit	qasper	20	100.0%	10.0%	383	769	824
vllm-metal	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	457	585	623

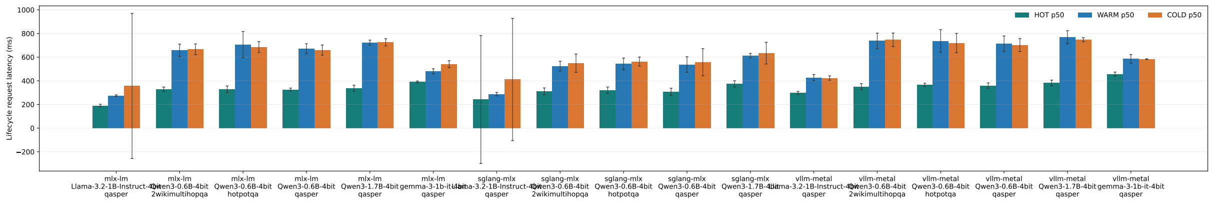

Figure 19: HOT, WARM, and COLD request p50 with p95–p50 upper whiskers. WARM recovery is reliable, but synchronous promotion remains visibly expensive.

Across cohorts, WARM/HOT p50 ratios range from 1.64 to 2.15. This changes the product interpretation: persistent native K/V is now a correctness-qualified backing representation, but request-path promotion is not yet economically qualified. Prefetch, overlap, and hot-set admission must remove that stall; quantized COLD additionally requires a model-and-workload quality gate.

The runtime now implements that next step without giving model-array mutation to background Python threads. An event-loop-owned coordinator deduplicates in-flight promotions by logical key, admits completed objects to a bounded hot set, and overlaps WARM reads with query-side work. Across five QASPER objects, MLX requires a 500 ms lead for 5/5 ready-at-demand and a 0.983 median demand/HOT ratio; SGLang reaches 5/5 by 350 ms and a 1.063 ratio. With no lead, the corresponding ratios are 2.123 and 1.561. All 70 paired outputs are exact. This demonstrates latency hiding under oracle lead, not a faster storage codec or a calibrated production predictor.

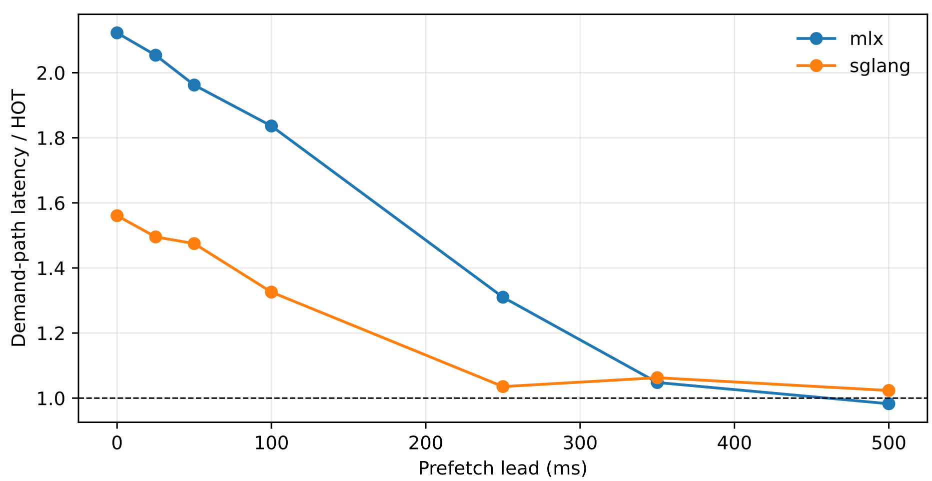

Figure 20: Scheduler-owned WARM promotion can hide request-path stall when useful lead is available. The dashed line is HOT latency.

The same native executors now traverse the HTTP streaming gateway. SGLang delivers 3.04–3.20 requests/s and MLX 1.56–1.66 requests/s over concurrency one through eight. Tail latency grows with the intentionally serialized model runners; cancellation after first token and cleanup succeed, with no retained request-owned native state across 36 exercised sessions. The model locks are a correctness boundary: an audit found that MLX cache captures were views into reusable arrays. Owning evaluated K/V arrays at the capture boundary restores HOT and WARM isolation and prevents later requests from mutating retained memory.

A separate MLX finite-wave curve extends physical-tier concurrency to 16. Every one of 124 HOT/WARM requests is baseline-exact. At concurrency 16, a shared WARM resource requires one promotion, sustains 2.91 requests/s, and avoids 630 MiB of duplicate K/V; 16 independent WARM resources require 16 promotions, read 1.26 GiB, and sustain 1.22 requests/s. This quantifies the benefit of immutable physical sharing while also exposing the queue cost of an unbatched, serialized runner.

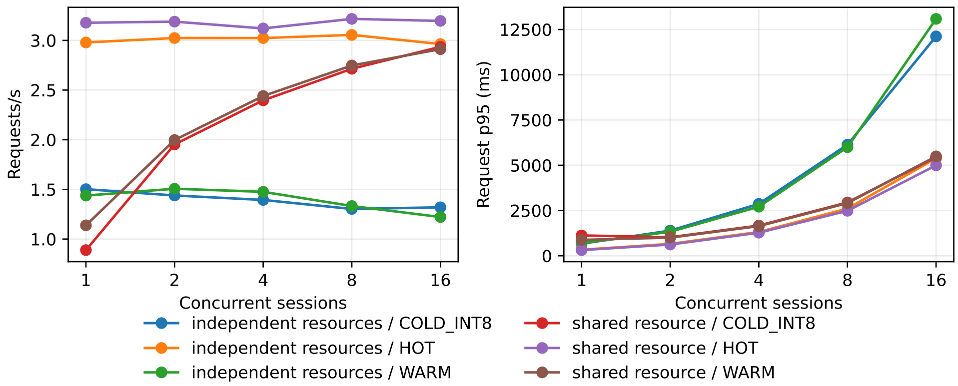

Figure 21: Shared and independent MLX resources through 16 concurrent sessions. The finite waves include promotion and serialized model queueing.

Long-session pressure adds 1,125 requests across QASPER, HotpotQA, and 2Wiki. With an eight-resource cyclic working set, HOT capacities two and four average 17 reloads per seed and reload every final revisit; capacity eight has zero reloads and evictions. Quality is invariant because selection is frozen. A 20-example QASPER selective-int8 study reaches only 4/20 exact generations when all K/V is quantized; restricting quantization to the late quarter reaches 14/20, still below the lossless product gate despite small aggregate F1 change.

A further 1,000-request QASPER matrix crosses HOT/WARM resource budgets $\{2, 8\}$ with local rotating windows $\{64, 256\}$. The small WARM budget produces 24–75% SOURCE starts, whereas WARM eight eliminates them. F1 is identical in all eight cells and window size has no material completion-tail effect. Full disk-backed WARM does not reduce resolve p95 relative to re-encoding these bounded sources, making reload-cost-aware admission a runtime requirement rather than an optional optimization.

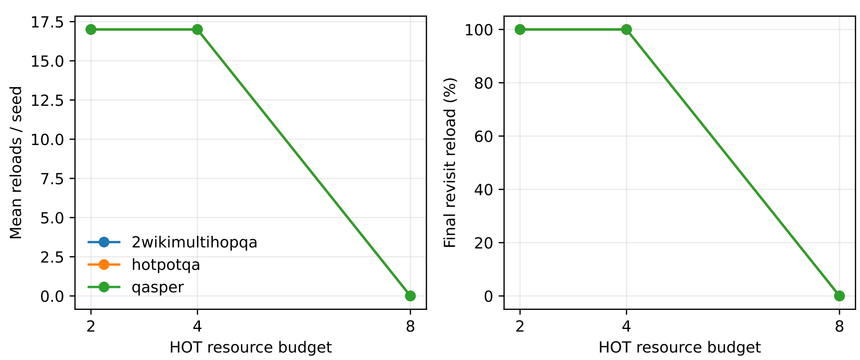

Figure 22: Sustained pressure exposes a working-set threshold rather than a quality effect: once all eight resources fit, reloads disappear.

9 Compatibility and Product Boundary

9.1 Common Qualification Contract

All engine rows now answer one question: at fixed or acceptably changed quality, when does PRA reduce total serving cost? The benchmark computes each query, candidate set, selected IDs, and selected intervals once. A cryptographic selection digest is then reused by `FULL`, `E0_SELECTED`, `E2_HOT`, and `E2_WARM`; optional E3 conditions add prefetch, remote WARM, COLD, or SOURCE restoration without rerunning retrieval. Context size, shared versus independent resources, and concurrency 1, 4, 8, and 16 are manifest dimensions. This separates retrieval quality from representation and transport.

Every row pairs task quality with serving measurements. Binary workloads derive successful requests per second as request rate times task success and report successful tasks per accelerator-hour. Monetary cost remains null unless an auditable hourly-cost source is supplied. Visible-token reduction and active-K/V reduction remain separate. Tail percentiles, physical memory components, transfer bytes and time, cache events, reuse, hardware metadata, status, and provenance are first-class fields. The current legacy registry has no row with both request rate and task success, so its quality-adjusted-throughput field is honestly `NOT_MEASURED` rather than reconstructed after the fact.

Figure 23 distinguishes a validated level from a deeper observed mechanism. In particular, AirLLM has a live E2 mechanism smoke but remains E0-qualified until natural quality and larger-than-VRAM evidence close the gate; SGLang has local E3-like HiCache controls but lacks distributed, scheduler-owned E3 validation. Raw throughput is deliberately absent from this maturity axis.

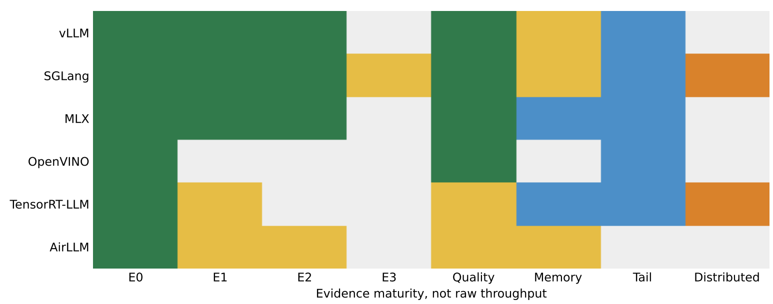

Figure 23: Cross-engine evidence maturity. Darker cells indicate stronger evidence; candidate mechanisms do not automatically promote validated level.

Table 17: Evidence-driven engine frontier. "Observed" may exceed "validated" when a mechanism exists but the corresponding quality, pressure, or scheduler gate remains open. Status words describe evidence, not numeric values.

Engine	Validated	Observed	Quality	Memory	Tail	Distributed
vLLM	E2	E2	natural	controlled	measured	not measured
SGLang	E2	E3	natural	controlled	model	candidate
MLX	E2	E2	natural	model	model	n/a
OpenVINO	E0	E0	natural	not measured	measured	n/a
TensorRT-LLM	E0	E1	controlled	measured	measured	candidate
AirLLM	E0	E2	controlled	controlled	not measured	n/a

Table 18: Implementation status on the measured branch and host.

Runtime	Status	Boundary
HF/PyTorch eager	measured	Native PRA model API and portable K/V primitives
Standalone gateway	contract tested	G00/G10/G01/G11, session lifecycle, prefix-preserving fallback, deltas, debug/close API, and HF-backed SSE
OpenAI-compatible HTTP	E0 implemented	Full-history safe default; typed engine profiles and explicit text fallback
DeepSeek/Pi agent bridges	contract tested	Typed event/RPC capture and explicit G10 fallback
`torch.compile`	blocked on host	API implemented; compiler/GPU gates recorded
Triton/custom CUDA	not run	Requires supported GPU and profiling evidence
vLLM	provider E0; companion E2	Default provider remains conservative; Paper 6 measures native APC and request isolation through concurrency eight
SGLang	provider E2	Companion Paper 6.1 measures native runner integration; local Radix-prefix and external-HiCache composition is controlled
MLX	provider E2	Companion Paper 6.2 measures native layer profiles, persistence, and concurrency; Paper 4.5 remains HF-centered
TensorRT-LLM	provider E0; native architectural	Companion Paper 6.4 measures CUDA selected-text load through concurrency 16; native attachment remains open
OpenVINO GenAI	provider E0	Companion Paper 6.3 measures CPU/GPU generation and continuous batching; the public native non-prefix attachment seam is blocked
FreeToken	E0/architectural	Remote fallback or planned provider; no native Paper 4.5 result

Table 19: Measured companion native-mechanism matrix. These rows calibrate engine realizations; they do not promote Paper 4.5's HF-centered profile evidence.

Engine/model	Hardware	Profile	Evidence tier	Status
vLLM-Metal V1 / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	all-layer native K/V + APC, concurrency 1–8	natural qa matched selection	840/840 exact at 0.6B/1.7B; lossless WARM 80/80; int8 COLD open
SGLang MLX / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	Radix + selected K/V + built-in HiCache file storage	natural qa matched selection	840/840 exact; lossless WARM 80/80; distributed HiCache open
MLX-LM / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	all-layer selected K/V, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 80/80; int8 COLD open
MLX-LM / Qwen3-1.7B	Apple M5 / 16 GB / Metal 4	FP/int8 all-layer selected K/V	natural qa routed evidence materialization	routed natural QA; bounded residency curve; oracle gap remains
MLX-LM / Llama-3.1-8B	Mac16,7 / 48 GB / Metal	all-layer native concat; reduced profiles calibration pending	natural qa matched selection	60/60 exact; warm ratio 1.032; cold ratio 0.935
vLLM-Metal V1 / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	all-layer native K/V + APC, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 5/5; int8 COLD 3/5 (smoke; gated)
vLLM-Metal V1 / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	global/local native K/V + APC, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 5/5; int8 COLD 5/5 (smoke; gated)
SGLang MLX / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 2/5 (smoke; gated)
SGLang MLX / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	mixed global/sliding-window topology	backend compatibility	blocked before PRA: per-layer window map unsupported
MLX-LM / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	all-layer selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 2/5 (smoke; gated)
MLX-LM / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	global/local selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 5/5 (smoke; gated)

Table 20: Bounded E0 serving matrix. These selected-text baselines do not imply native non-prefix K/V support.

Engine/model	Hardware	Profile	Evidence tier	Status
vLLM CUDA V1 / TinyLlama-1.1B-Chat-v1.0	NVIDIA RTX 5060 Laptop / 8 GB / WSL2	selected-text E0 + APC, concurrency 1–16	serving bounded load context pressure	all controlled outputs exact; selected C16 91.9–93.5 req/s; native E2 open
TensorRT-LLM 1.2.1 / TinyLlama-1.1B-Chat-v1.0	NVIDIA RTX 5060 Laptop / 8 GB / WSL2	selected-text E0, concurrency 1–16	serving bounded load context pressure	all controlled outputs exact; selected C16 79.6–81.9 req/s; native E2 open
OpenVINO GenAI 2026.3.1 / Qwen2-0.5B-Instruct-int4-ov	Intel i7-1355U / Iris Xe	selected-text E0, continuous batching 1–16	serving bounded load context pressure	selected controlled recovery exact; C16 8.4–14.2 req/s; native E2 blocked

The engine registry now includes bounded-load E0 rows for vLLM CUDA, TensorRT-LLM, and OpenVINO GenAI. On the shared RTX 5060 Laptop GPU, 66-token vLLM selection sustains 91.9–93.5 requests/s at concurrency 16 as the matched full prompt grows to 1,274 tokens; independent full-context throughput falls to 10.0 requests/s with 1.41 s TTFT p99. TensorRT-LLM reaches 79.6–81.9 requests/s for its 66-token selected condition at the medium pressure point. OpenVINO selection remains exact through concurrency 16 on Iris Xe, while full-context correctness varies with distractor pressure. These are engine realization measurements at E0. They strengthen the selected-text baseline that native E2 must beat, but do not promote any CUDA or OpenVINO row to E2.

Table 21: Cross-engine context pressure at concurrency 16 and the largest tested prompt. Ratios are normalized within engine; values above one favor selected text.

Engine	Workload	Sel./full tok.	Succ. S/F	Req/s S/F	Req. ratio	p99 S/F	Tail ratio
vLLM CUDA	Shared	66/1274	1.00/1.00	92.4/72.1	1.28×	46/68	1.46×
vLLM CUDA	Independent	66/1274	1.00/1.00	91.9/10.0	9.19×	47/1408	29.83×
TensorRT-LLM	Shared	67/1276	1.00/1.00	80.7/56.9	1.42×	58/149	2.57×
TensorRT-LLM	Independent	67/1276	1.00/1.00	80.3/52.6	1.53×	58/173	2.98×
OpenVINO	Shared	63/831	1.00/1.00	14.1/11.5	1.23×	235/483	2.06×
OpenVINO	Independent	63/831	1.00/0.82	10.8/10.2	1.06×	319/530	1.66×

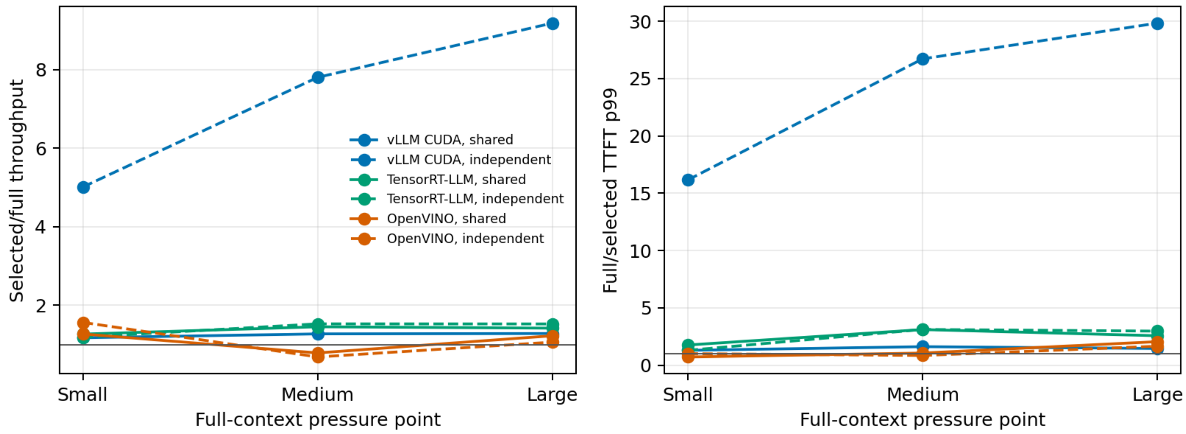

Figure 24: Within-engine throughput and TTFT-tail ratios over three context pressure points. Absolute rates are not compared across dissimilar hardware.

At the largest point, selected text improves shared-resource throughput by 1.23–1.42× across all three engines and reduces TTFT p99 by 1.46–2.57×. Independent resources expose the largest difference on vLLM CUDA: selected/full throughput is 9.19× and the TTFT-tail ratio is 29.8× after the long-prefix scheduling boundary is crossed. TensorRT-LLM shows a steadier 1.53× throughput and 2.98× tail advantage. OpenVINO's independent full condition recovers only 0.82 of targets at the largest point, so its 1.06× throughput ratio must be read with the reported quality difference. The comparison therefore establishes a portable pressure-test method and a strong E0 baseline, not a hardware ranking.

Table 22: Frozen matched E0 selected-text versus E2 native-K/V quality. Each dataset contains 20 routed examples; selection is fixed before cold, warm, multi-query, and concurrency-eight engine consumption.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.079/0.079	100.0%	100.0%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.074	100.0%	100.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.101	100.0%	100.0%

Table 23: Macro E2/E0 execution-cost ratios; lower is better. Cold includes one-time ingestion, warm and multi-query use median completion, concurrent uses inverse requests/s, and E2 usable is milliseconds.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	0.985	1.004	0.993	0.981	86.0

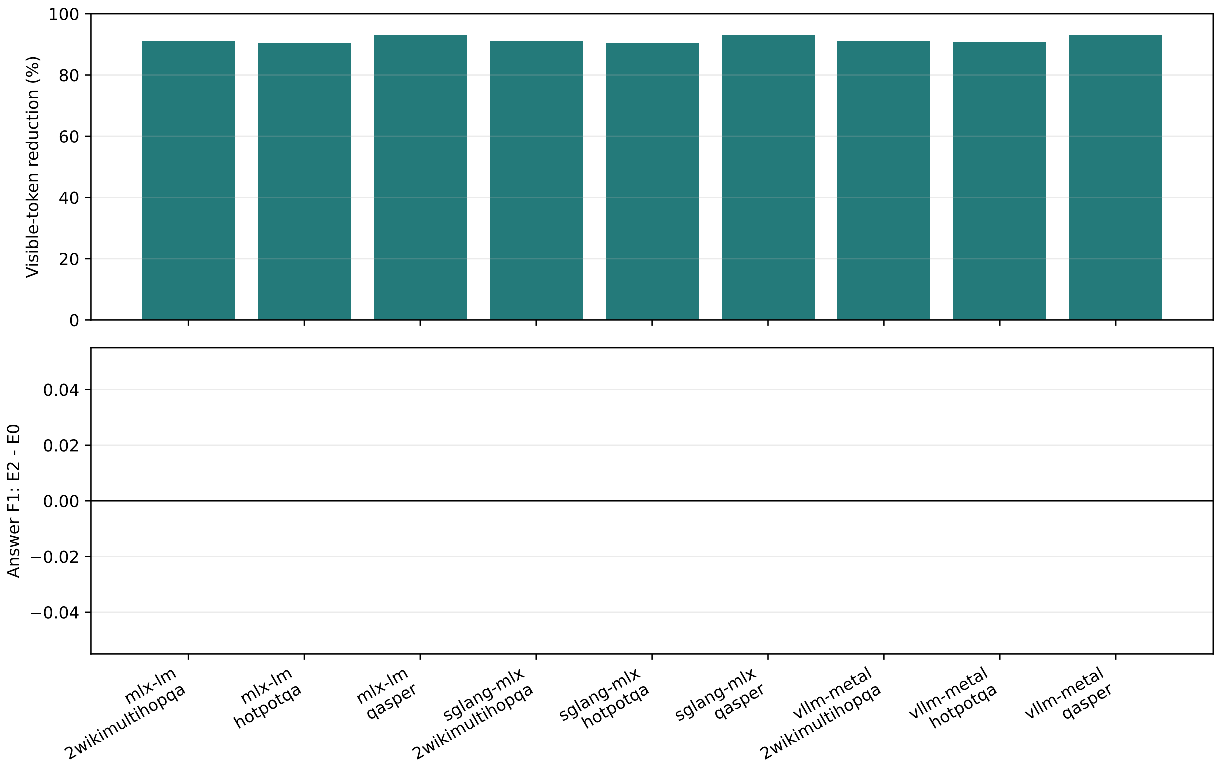

Figure 25: Holding routed evidence fixed separates visible-token reduction from semantic preservation. After restoring E0 through APC and capturing E2 from the same source prefill, all three engines retain every paired output and F1.

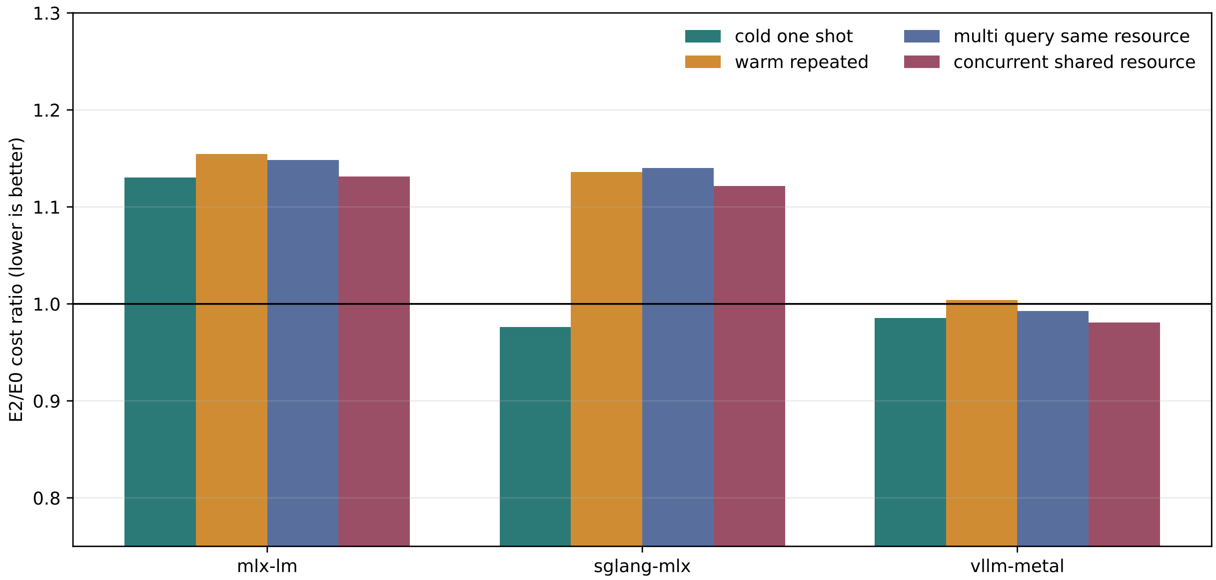

Figure 26: Representation economics after freezing retrieval. Ratios below one favor native E2. The current SGLang path wins only cold ingestion-inclusive cost; vLLM is near parity; MLX and SGLang still pay unfused decode overhead in reused and concurrent regimes.

The matrix now distinguishes companion evidence from generic provider defaults. On vLLM-Metal, selected native pages recover 75/75 answers and reach 34.41 requests/s at concurrency eight; after the first request in a native-selection namespace, APC supplies 128–144 prompt tokens. Alternating selected and ordinary rows satisfy their expected outcome in 40/40 cases. On SGLang-MLX, an 81-token shared-prefix hit composes with external HiCache: selected A/C recover 10/10, cleanup B recovers 0/5, and every selected request contains exactly one memory copy. A separate five-seed bridge uses SGLang's built-in file-storage backend: fresh adapters hit and recover 5/5, with 38.7 ms mean writes, 122.2 ms mean cold L1 restoration (36.2 ms median), and 0.013 ms warm lookup. This is a local backend-contract result, not distributed HiCache. On MLX-LM, routed ordinary and native K/V agree on 60/60 original-answer QA pairs; evidence recall@4 ranges from 0.348 to 0.775, so the companion result retains an explicit open oracle gap. These are pinned controlled realizations, not portable claims that the default SDK provider may advertise unconditionally.

The shared matched benchmark makes the product boundary sharper. MLX-LM and SGLang-MLX preserve all 840 E0 outputs while moving 90.6–93.0% of visible prompt tokens into native K/V. Their current selected-cache wrappers remain 12–16% more expensive in reused or concurrent regimes. vLLM-Metal also preserves 840/840 outputs with zero pairwise F1 change. Its four macro cost ratios lie between 0.981 and 1.004. An earlier 593/840 result compared full-prefix E0 prefill against query-only E2 prefill; a five-example audit shows that APC-restored E0 and native E2 match 5/5 while full-prefix and APC-restored E0 themselves differ in 2/5. The SDK therefore treats all three pinned paths as semantic mechanism passes while keeping their economic status experimental. Engine labels alone never select these capabilities.

The expanded confirmation raises each engine to 149 unique questions and 2,086 paired requests. Every engine is 2,086/2,086 exact, for 6,258/6,258 pairs overall; the pooled Wilson 95% lower bound is 0.9994. Visible-token reduction is 90.4–93.0%. The macro ratios in Table 24 show why semantic and economic gates remain separate: vLLM-Metal is within 0.9% of parity in every regime, SGLang-MLX pays 5.4–7.4% on reused paths, and MLX-LM pays 7.2–13.9% while using the same frozen evidence.

Table 24: Expanded selector-frozen E2/E0 cost ratios over 149 unique questions and 2,086 paired requests per engine. Lower is better.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.043	1.139	1.133	1.072	109.9
sglang-mlx	0.980	1.074	1.071	1.054	117.1
vllm-metal	1.009	0.997	1.003	0.991	149.7

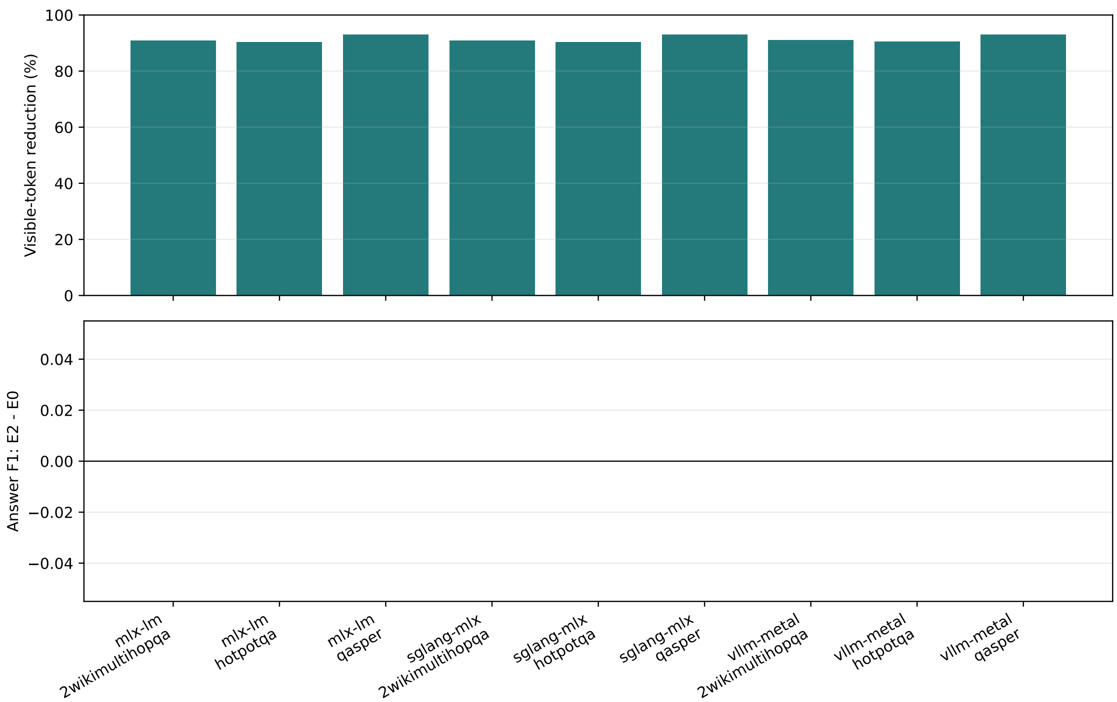

Figure 27: Expanded frozen-selection quality and visible-token accounting. Every engine preserves every paired output; confidence reflects finite sample size.

The vLLM audit also establishes a product invariant that is easy to miss at an HTTP boundary. APC hashes visible prompt state, but native PRA evidence is hidden from that state. A native provider must therefore derive an opaque APC salt from tenant scope, ordered logical-memory identity, selected-token count, position policy, and consumer layers. Stable selections retain warm reuse; changed selections cannot inherit query K/V produced under another memory. The SDK exposes this derivation as an engine helper and tests equal-selection reuse, changed-selection isolation, geometry sensitivity, and identifier non-disclosure. A five-example capture/replay audit preserves E0's first token in 5/5 cases at the nominal position base. Holding source-ingestion and query-prefill geometry fixed raises complete 24-token parity from 3/5 to 5/5; the full corrected cohort then reaches 840/840.

The product surface includes installable package metadata, Hugging Face loading, runtime config serialization, direct and external reference registration, session isolation, callable and folder-backed capability registration, lazy view activation, typed resource discovery, safe execution, compact result ingestion, selected replay, address search, cursors, size-adaptive native result routing, lazy native-region promotion, generation, inspection, benchmark commands, and two executed notebooks. The canonical pra CLI exposes separate `model`, `adapter`, `profiles`, `bundle`, `runtime`, `agent`, `gateway`, and Hub-only `hf` namespaces. `pra-hf` remains a deprecated alias for one release cycle. The `pra gateway serve` command launches the independently testable gateway with health, capability, logical-request, and OpenAI-compatible endpoints. A deployment can therefore discover unsupported paths before serving.

9.2 Model onboarding, calibration, and community distribution

The product workflow treats architecture adaptation, learned routing adaptation, profile calibration, engine realization, and Hub distribution as different operations. `ModelInspector` reads configuration metadata without loading weights. `StructuralAdapterBuilder` emits a versioned declarative mapping for conventional Qwen, Llama, and Gemma decoder attention. `ModelValidator` records V0–V9 independently, so discovering Q/K/V projections cannot be reported as semantic success. The later stages cover disabled-PRA parity, native capture, visible-prefix equivalence, source-relative position geometry, cached decode, grouped-query layout, selected-region consumption, and generation.

`ProfileCalibrator` packages measured rows from the existing benchmark registry. A small-cohort optimum remains `QUALITY_MAX_CANDIDATE` with SMOKE/`CALIBRATION_PENDING` evidence; only benchmark-scale validation may assign `QUALITY_MAX`. `PRAModelBundle` then combines immutable base model identity, structural and learned adapters, profiles, evidence, runtime compatibility, engine realizations, and provenance without copying base-model weights. Hub pull and push are optional transport operations under `pra hf`; generic Transformers execution does not depend on Hub login.

HF model → inspect → structural adapter → semantic validation → optional learned adapter
→ profile calibration → PRA bundle → runtime/agent/engines → optional Hub publication

Figure 28: The model-onboarding path. Each arrow is a separately inspectable service boundary rather than an implicit side effect of one CLI callback.

9.3 Engine-neutral runtime providers

The canonical launch path is `pra runtime serve` with a model and `-e ENGINE`. Click parses the request, `RuntimeManager` resolves a provider, and that provider translates a common model, revision, PRA bundle/profile, device, session, and cache contract to the upstream engine. Repeated `--engine-arg KEY=VALUE` values preserve engine-specific escape hatches without cloning upstream CLIs. Providers return serializable handles and distinguish process start from readiness. Inspection reports static declarations; doctor checks report installed dependencies and live endpoints separately.

`pra runtime serve` → `RuntimeProvider` → {HF, vLLM, SGLang, MLX}
agent → [optional gateway] → runtime → engine

Figure 29: One runtime command preserves the separation between agent policy, optional gateway mediation, and engine-native execution.

The default vLLM provider deliberately advertises E0 unless the measured native bridge is installed: Paper 6 now establishes a controlled Metal V1 generation path with APC and offline concurrency through eight, but not a portable CUDA or production online-scheduling realization. SGLang and MLX advertise the companion papers' measured native mechanism support under their pinned environments. Paper 6.1 additionally closes local shared-prefix/HiCache composition at the runner boundary, while distributed HiCache and E3 scheduler-aware PRA remain false. The shared capability object feeds runtime inspection, gateway negotiation, agent launch, and benchmark records.

9.4 Agent-level configuration and interactive surfaces

Low-level `pra.yaml` controls model/context execution. A separate, versioned agent profile controls model endpoint, workspace, durable sessions, tool approval, skill directories, MCP declarations, task policy, and generation budgets. Resolution is explicit CLI options, selected named profile, default profile, project config, user config, then package defaults. The short option `-p` selects an agent profile; `-P` independently overrides PRA. Credential files remain server-side references and are redacted from session, inspection, benchmark, and model-card output.

Agent profiles now independently select `context.transport=auto|pra|text`, text-fallback permission, and required features. The CLI exposes `--context-transport` and `--allow-text-fallback/--no-text-fallback`; `-P/--pra` continues to select the model/runtime PRA profile and is not overloaded with wire semantics. The launch summary, TUI `/runtime` command, and web session diagnostics report requested and resolved transport, capability source, gateway mode, integration level, resource-delta support, and native-K/V status without including record bodies.

`pra agent chat` is the stable TUI reference and can resolve a default profile without model flags. `pra agent run` provides a noninteractive form. The optional FastAPI/WebSocket UI is another presentation layer over the same `PRAAgent`, `AgentLauncher`, session, task, and authorization services. It supports multiple durable conversations and sequence-based replay after reconnect. A dangerous tool call remains blocked until the backend receives a typed one-shot approval; the browser cannot bypass host policy. The UI binds to loopback by default and is experimental rather than a scientific contribution or production security claim.

10 Limitations and Next Gates

The current evidence is intentionally incomplete. The local CUDA GPU is old, and the companion serving-engine measurements run on one Apple host. We have not measured continuous batching, request-level p95/p99, asynchronous transfer overlap, a production page allocator, prefetch, multi-session contention, or fused attention preparation. The LRU fixture validates byte accounting but is not a production cache policy study. The inherited language-model cohort is too small for quality claims. The new capability and compact-result workflows and Paper 8 task/session integration are executed contract tests, not a new latency or answer-quality experiment. The shell is intentionally single-agent; subagents and learned long-term memory remain scoped to Papers 9 and 10. Model-resident result routing currently requires an isolated model session; safe concurrent multi-tenant partitioning of model-resident result K/V remains an engine-level gate. The multi-axis HF bridge serializes high-level generation on one wrapped model to isolate temporary adapter state. It is a correctness reference, not a continuous-batching implementation. The cache key now rejects physical reuse across changed source revisions, positions, materialization layouts, or scopes, and a concurrent fixture covers logical tenant isolation, pressure, and cancellation. It does not establish hard GPU partitioning or production multi-tenant throughput. Phase selection and portable E2/E3 engine integration remain open. The G01 parser recognizes explicit system, tool, and attachment boundaries; it does not claim semantic recovery from arbitrary opaque traffic.

The Paper 4.5 gateway and agent-transport profiles are deterministic transport experiments. They do not use a real serving engine or distributed scheduler, and therefore does not measure physical prefix-cache hits, worker affinity, warm-prefix TTFT, continuous batching, index residency, or network cost. The reported reuse fraction counts exact serialized messages, not tokenizer tokens. Engine profiles are conservative defaults and require runtime probing or explicit configuration before stronger capabilities may be advertised. Companion engine studies do not change this profile's evidence tier and are intentionally kept outside Paper 4.5's HF-centered metric tables.

The cross-model cohort contains only three one-token continuations per model. It is sufficient to expose exact disabled behavior, K/V topology, position, lifetime, and coarse semantic drift, but not to estimate broad language quality. The Llama row uses a public weight mirror because the pinned official repository was inaccessible. Gemma is partial rather than passed: its global-only adapter cannot make external K/V visible to unchanged local sliding layers. Sparse profiles are declared per family and are not assumed to preserve quality. The layer calibration reuses that diagnostic cohort. Its 102 rows vary profiles, not independent examples, so they identify gross transport drift and set conservative SDK defaults but cannot estimate downstream answer quality. Qwen last-24's target-token gain is especially vulnerable to this small-cohort selection effect. It is consequently exposed only as `QUALITY_MAX_CANDIDATE` with pending calibration, not as `QUALITY_MAX`. Paper 3 owns retrospective placement evidence on MuSiQue and 2Wiki; a future calibration should add held-out QASPER, HotpotQA, typed-result, and long-generation workloads before any sparse profile becomes a default.

The next experiment should move to a compute-capability 8.0 or newer GPU with a pinned software stack. It should first reproduce eager parity, then measure `torch.compile` cold and warm paths. Only measured hot spots should move to Triton. Layout and hierarchy sweeps should cross contiguous versus indexed selection, whole chunks versus intervals, GQA head count, batch heterogeneity, and warm-cache locality. End-to-end runs should report TTFT, inter-token latency, throughput, peak HBM, off-device bytes, temporary bytes, transfers, and reload amplification together.

The next policy experiment should use a trained model and retrieval workload, hold selected-token budgets fixed, and compare answer quality, TTFT, inter-token latency, selection churn, and routing-layer disagreement. The implemented phase/shared handoff closes the cache-correctness prerequisite; its remaining gate is workload-scale calibration rather than mechanism availability.

Deeper engine integration remains outside Paper 4.5. Companion engine papers have established SGLang and MLX native mechanism baselines and controlled vLLM-Metal V1 generation. A shared matched benchmark now closes E0/E2 semantic parity for all three engines. The corrected vLLM-Metal geometry reaches 3,360/3,360 exact cross-family pairs, while expanded Qwen confirmations preserve 2,086/2,086 per engine. SGLang and MLX still expose unfused decode overhead on reused paths. Those results remain environment-specific and do not turn the generic provider into an unconditional E2 claim. Native APC and offline concurrency now pass on the pinned Metal path when cache identity includes native selection and geometry; CUDA replication, online queued concurrency, distributed placement, broader positions, masks, GQA, and heterogeneous batching must pass before portable speed comparisons. If useful behavior requires semantic residency, retrieval-aware scheduling, or cross-request PRA block sharing, that work belongs to the deep engine paper.

11 Conclusion

PRA's selected-memory fraction is an algorithmic fact, not a systems result. This paper turns that distinction into code and measurement. One runtime now connects the public model API, authenticated memory lifecycle, typed resources, lazy tool and skill disclosure, safe execution, compact result backing, selective replay, durable tasks and sessions, physical K/V planning, caching, profiling, and backend handoff. Typed results build full native indexes only within configurable token/byte ingestion budgets; larger records retain cheap recovery interfaces and promote selected regions lazily. Its policy lattice makes routing frequency and physical lifetime inspectable, while the gateway permits ordinary and PRA-aware agents and engines to coexist without sending native tensors over the logical wire. The agent no longer flattens selected records before remote execution: AUTO preserves typed context through G10, G11, and direct PRA endpoints, while an ordinary endpoint receives one deterministic text rendering. In the five-turn task/tool fixture, PRA-DELTA cuts wire bytes by 29.6% relative to

TEXT and sends 8 rather than 24 resource bodies, without changing selected identity or task metadata. At 50 turns, including reconnect and resynchronization, the reduction grows to 74.6% versus TEXT and 80.7% versus PRA-FULL. A ten-example MLX-LM QASPER cohort then preserves exact model input, output, and gold-answer log-probability across TEXT, PRA-FULL, and PRA-DELTA in 10/10 cases. The protocol result is therefore behavioral as well as structural. Conventional sequential-prefix K/V and detached PRA resource/native state are now separate capability and lifecycle axes. The tested gateway preserves every eligible prior prefix in its five-turn fixture and sends typed message/resource deltas through a prepared session, while correctly leaving physical cache hits unknown. The HF semantic reference now passes full-prefix logit equivalence on the tested Qwen and Llama checkpoints, plus record-boundary deduplication and request-long decode-lifetime gates. Gemma validates native MQA mechanics but identifies a global/local topology boundary rather than a full semantic pass. Layer placement is now a versioned calibration result rather than an implicit "PRA layer" convention. Separate address, detail-K/V, routing, and consumer sets make persistent storage and request-time computation independently auditable; missing detail can never trigger a silent profile downgrade. The eager primitives are exact and inexpensive in isolation, but off-device transfer multiplies their cost on the measured host. Compilation remains a negative gate. Companion engine runs establish pinned native realizations but still leave portable E2/E3 integration unmeasured. The practical conclusion is conservative: cache locality, physical residency, and context lifecycle deserve attention before kernel novelty, and every future speed claim must match semantics while accounting for the whole request.

A Reimplementation Guide

The stable public entry points live in `src/pra_hf/runtime.py`. A reimplementation needs five small contracts. First, represent warm K/V as equal-shaped tensors $[B, H_{kv}, T, D]$. Second, express selections as stable URI, layer, and half-open token intervals. Third, merge overlaps before applying a hard global token budget. Fourth, pack selected tensors by consumption layer without expanding grouped-query heads. Fifth, emit stage times and physical bytes independently of routing-quality metrics.

The model adapter only needs `add_reference`, `generate`, and `inspect`. The thin serving request contains selected identities and materialized-token metadata, never semantic scores. Tool execution receives a separate authorization object after discovery. These boundaries allow another runtime to reproduce the interface without adopting the Python implementation.

Paper 8's finalized geometry adds one further invariant. A frozen selection is an immutable set of source-local anchors. Expansion may change the interval width or choose the complete selected record, but it cannot cross record boundaries or change source identity. Overlap is merged before budgeting, and each consumer resolves the same logical interval to that layer's own projected K/V. The query starts after the longest source extent, while separate records retain source-relative positions. Raw intervals are retained for diagnostics; the canonical union is the only geometry materialized. The named `paper3_default`, `paper7_selected_detail`, and `paper8_full_record_diagnostic` profiles resolve to ordinary runtime configuration. The public sequence is `route`, `freeze_native_selection`, `plan_native_materialization`, then `generate_with_native_plan`. Task operations follow the analogous control boundary: parse untrusted proposals, validate them against a copy of the task graph, and only then replay accepted events through the session service.

A.1 Agent transport implementation map

The semantic/wire split is implemented in `src/pra_hf/agent_transport.py`. `AgentTurnContext` keeps messages, context records, tool records, skill records, provider-native tool schemas, and task metadata independent of transport. `context_record_to_wire_resource` is the only logical projection; `render_text_messages` is the only text compatibility renderer. `CapabilityNegotiator` distinguishes a reachable ordinary server from an unavailable endpoint and caches the version-1 feature handshake. `NegotiatedRemoteBackend` resolves the mode, computes message and resource deltas, sends the OpenAI-compatible envelope, and clears its acknowledged inventory on refresh.

The call site is `src/pra_hf/agent.py`. `_turn_context` separates conversation records from detached records and projects active task state; `_generate_turn` delegates either to the negotiated backend or the canonical embedded text renderer. Thus selection and tool authorization remain agent responsibilities, while rendering and network serialization remain transport responsibilities.

The public wire dataclasses and protocol-version check are in `src/pra_hf/deployment.py`. Gateway capability composition and G10/G11 mediation are in `src/pra_hf/gateway.py`; acknowledged message/resource inventory, body-free unchanged operations, and resynchronization state are in `src/pra_hf/gateway_session.py`. The compatibility matrix is executable in `tests/test_agent_transport.py`; the five-turn byte fixture is `experiments/paper4_5_runtime/run_agent_transport_protocol.py`.

A.2 HF native-reference implementation map

The four-role resolver is in `src/pra_hf/layer_profiles.py:1--430`. Lines 27–77 define independent encoding and missing-detail policies; lines 81–178 normalize all, suffix, fractional, evenly spaced, and explicit profiles; lines 181–244 validate the L_A, L_{KV}, L_R, L_C invariants and serialize provenance. The stable public fields and registry precedence are applied in `src/pra_hf/config.py:19--420`. The versioned model entries live in `src/pra_hf/model_profiles/layer_profile_registry.json`.

Lines 328–510 of the HF `injection.py` module separate reference payloads. Every injected layer can produce compact identity metadata for cross-layer mapping, address layers retain their search representation, and only detail layers retain native K/V. The cache record exposes `has_detail_kv` in `src/pra_torch/memory.py:99--140`; materialization rejects an address-only chunk before attention in `src/pra_torch/core.py:232--270`. Selecting a chunk ID at L_R never licenses reuse of L_R's tensor at L_C: the identity is resolved to the destination layer's independently projected K/V.

The complete geometry is concentrated in `src/pra_hf/native_geometry.py:59--374`. Lines 59–107 define four materialization modes and the three named research profiles. Lines 130–169 make raw and unique interval accounting, source lengths, and query offset part of the immutable plan. Lines 189–238 clamp expansion to one record; lines 310–374 normalize overlap before slicing each consumer layer's own K/V.

The high-level path is in `src/pra_hf/model.py:932--1191`. `plan_native_materialization` separates routing anchors from selected chunk, asymmetric window, selected-record, or full-scope geometry. `native_next_token_logits` is the E0 diagnostic. `generate_with_native_plan` is the public fixed-plan path, and `debug_decode_with_native_plan` spells out prefill, local cache handoff, and one-token decode calls. All three use `src/pra_torch/hf/injection.py:494--529` to start the direct query after the longest active source extent.

At attention time, `src/pra_torch/hf/qwen.py:340--352` prepends selected native K/V to local K/V and records compact request-lifetime state before the family-native eager kernel. Llama and Gemma 3 inherit this shared path while overriding only family symbols and native scope. The trace owner is defined in `src/pra_torch/hf/adapter_base.py:107--129` and aggregated at `src/pra_torch/hf/injection.py:228--241`; no full K/V or attention tensor is retained for telemetry.

Portable streaming begins in `src/pra_hf/deployment.py:227--341`. The worker owns logical references, the cancellation criterion, and cleanup. Gateway mediation and SSE framing are separate at `src/pra_hf/gateway.py:131--231`, so a transport disconnect closes the iterator without exposing backing content. Tenant-scoped cache identity and local eviction are in `src/pra_hf/runtime.py:568--672`. Cache identity includes source revision, positional and materialization signatures, and scope; equality is the reuse predicate rather than a heuristic URI match. Engine capability fields separately expose interval materialization, request lifetime, residency, tenant isolation, phase selection, and scheduler hints. An independent implementation should fail a required unsupported field before reference registration.

A.3 Gateway session implementation map

The independent capability axes begin at `src/pra_hf/deployment.py:38`: the capability record carries engine type, PRA level, prefix-cache mode, session state, incremental-message support, resource deltas, affinity, and explicit handles. Typed request serialization starts at line 129, and the profile-parameterized OpenAI adapter at line 242. The versioned defaults and validation are in `src/pra_hf/engine_profiles.py:1--90`; adding an engine requires an enum member and one conservative registry row before any feature override.

Ephemeral state is isolated in `src/pra_hf/gateway_session.py:186--434`. `resolve_turn` compares explicit full or delta history, derives resource operations, and computes the affinity key. `commit` advances fingerprints only after a successful decode. It stores resource IDs, digests, and metadata, never content tensors. The gateway execution order is visible at `src/pra_hf/gateway.py:147--457`: resolve and invalidate, prepare once, transform G00–G11, execute, commit trace state, and close explicitly. HTTP capability, session-debug, and session-delete handling follows at lines 460–538. The same registry entry points are surfaced by `src/pra_hf/runtime.py:1194--1220`, preventing the HTTP process from becoming the sole owner of session semantics.

A.4 Execution and deployment code map

Line references identify the artifact version accompanying this paper. `src/pra_torch/execution.py:61--107` defines the four policy axes and their validation; lines 358–425 define tensor-free identities and row-local plans; lines 428–504 own request epochs, churn, and trace summaries; and lines 507–632 separate selection reuse from opaque materialization lifetimes. The important invariant is the type boundary: a selection controller returns URI/chunk spans, while only an engine resolver may return tensors or block handles.

The canonical typed-result admission policy is implemented at three layers. `src/pra_hf/adaptive_context_runtime.py:82--236` defines global and per-record token/byte limits, inheritance, deferral, and serialization. `src/pra_hf/progressive_context.py:58--66` defines the four lifecycle states; lines 347–391 define the public audit and lazy-region records; and lines 602–773 perform idempotent size-gated full indexing and selected-region native encoding. The facade in `src/pra_hf/runtime.py:1119--1268` applies automatic preparation at ingestion and exposes preparation, lazy promotion, and lifecycle inspection. This order is important: policy is resolved before model-native construction, and a gate rejection preserves the backing identity and recovery interfaces.

`src/pra_hf/hf_execution.py:49--174` is the family-independent HF bridge. At the routing layer it turns the current hidden representation into a logical plan. For shared scope it remaps the source-layer hits through `map_chunk_identities_to_layers`; each destination therefore receives its own native grouped-query K/V. The public request lock and precedence entry point are in `src/pra_hf/model.py:710--895`. Qwen, Llama, and Gemma call the same bridge from their thin attention adapter.

For a distributed implementation, begin with `src/pra_hf/deployment.py:100--174`: reproduce that JSON request without adding tensor fields. At lines 175–182, expose capabilities, session setup, generation, streaming, and close as separate methods. Then reproduce deterministic mediation in `src/pra_hf/gateway.py:24--178`: negotiate first, transform according to G00–G11 second, and append component-local trace events last. The HTTP process starts at line 180; the CLI binding is `src/pra_hf/gateway_cli.py:35--59`. This order keeps an unsupported native request from being mistaken for successful text fallback. The agent bridge implementation is `src/pra_hf/agent_plugins.py:1--253`; its family-specific parsers terminate in the common logical request builder rather than engine calls.

Frozen interval expansion and layer-native slicing are in `src/pra_hf/native_geometry.py:1--374`; the SDK facade delegates through `src/pra_hf/runtime.py` so users do not access adapter internals.

B Artifact Map

Artifact	Role
`src/pra_hf/runtime.py`	Unified facade, plans, materializer, cache, profiler, HF adapter, thin vLLM contract
`src/pra_hf/runtime_providers.py`	Provider registry, canonical runtime configuration, conservative capabilities, managed handles, readiness, and upstream launcher translation
`src/pra_hf/onboarding.py`, `src/pra_hf/bundle.py`	Structural inspection/adaptation/validation, profile packaging, portable bundles, and the optional Hub boundary
`src/pra_hf/cli.py`	Thin canonical `pra` command tree and deprecated `pra-hf` aliases
`src/pra_torch/execution.py`	Engine-neutral policy axes, logical plans, precedence, lifecycle, and analytical routing count
`src/pra_hf/hf_execution.py`	Request-owned token/shared and token/per-layer HF execution bridge
`src/pra_hf/deployment.py`, `src/pra_hf/gateway.py`	JSON wire types, engine capabilities, E0/E1 adapters, G00–G11 mediation, and HTTP process
`src/pra_hf/engine_profiles.py`, `src/pra_hf/model_profiles/engine_profile_registry.json`	Versioned engine/cache/session defaults with conservative capability resolution
`src/pra_hf/gateway_session.py`	Tenant/session/model-isolated ephemeral engine handles, canonical and serialized prefix state, resource versions, history/resource deltas, affinity, and invalidation
`experiments/paper4_5_runtime/run_gateway_session_profile.py`	Five-turn prefix/session contract profile, two-cache architecture, traces, tables, and figures
`experiments/paper4_5_runtime/run_agent_transport_scaling.py`	Deterministic 20/50-turn TEXT, PRA_FULL, and PRA_DELTA wire scaling with resynchronization and body-transmission accounting
`experiments/paper4_5_runtime/run_agent_transport_model_backed.py`	Held-out model-backed G10 transport parity, task/tool visibility, gold-answer log-probability, TTFT, and execution-message checks
`experiments/paper4_5_runtime/build_product_matrix_v2.py`	Strict engine–model–profile–hardware product matrix with explicit evidence tier, per-metric status/provenance, qualification level, and missingness fields
`experiments/paper4_5_runtime/build_engine_qualification.py`	Selector-frozen E0/E2/E3 manifest, evidence-driven maturity registry, table, and cross-engine figure
`experiments/paper4_5_runtime/run_storage_policy_trace.py`	Weighted-versus-LRU retention trace with task/shared reuse, reconstruction, and estimated context-cost accounting
`docs/papers/shared/results/paper4_5_runtime/prefix_preservation_results.csv`	Canonical per-turn logical-prefix and transport measurements; physical hits remain unknown
`src/pra_hf/agent_plugins.py`	DeepSeek Harness and Pi typed-event bridges
`src/pra_hf/native_geometry.py`	Frozen record-bounded intervals and layer-native K/V plans
`experiments/paper4_5_runtime/run_execution_policy_profile.py`	Five-seed multi-axis mechanism profile and trade-off figure
`experiments/paper4_5_runtime/run_cross_model_validation.py`	Restartable Qwen/Llama/Gemma semantic gates, physical accounting, reuse and multi-tenant controls, and cross-model figure

Artifact	Role
experiments/paper4_5_runtime/run_layer_profile_calibration.py	Corrected cross-model contiguous/sparse profile sweep, storage accounting, registry selection, and quality–cost figure
src/pra_hf/layer_profiles.py	Four-role layer schema, normalized selection, registry precedence, lifecycle, and missing-detail policy
src/pra_hf/profile_benchmarks.py	Versioned model/workload profile registry, normalized metrics, missingness status, and resolution trace
src/pra_hf/model_profiles/pra_profile_benchmarks.json	Packaged semantic and physical profile registry
experiments/paper4_5_runtime/build_profile_matrix.py	Deterministic registry and manuscript-table generator
src/pra_hf/runtime_benchmark.py	Structured eager/compiled and hierarchy microbenchmark
src/pra_hf/agent_resources.py	Typed resource identities and discovery
src/pra_hf/agent_execution.py	Schema validation and independent host authorization
src/pra_hf/agent.py	Task-aware turn orchestration and persistent agent facade
src/pra_hf/agent_profiles.py	Versioned named profiles, precedence, credential redaction, runtime mapping, and shared agent launcher
src/pra_hf/agent_web/	Optional FastAPI/WebSocket multi-session presentation, reconnect replay, one-shot approval, and safe detached lifecycle
src/pra_hf/session_service.py	In-memory and atomic local logical session persistence
src/pra_hf/task_context.py, src/pra_hf/task_scope.py	Versioned task DAG, provenance, scope admission, and working-set lifecycle
src/pra_hf/toolsets.py, src/pra_hf/tui.py	Reusable tools and the resumable coding-agent terminal
src/pra_hf/capability_sdk.py	Callable and skill-folder ingestion, bounded palettes, and exact capability activation
src/pra_hf/typed_context.py	Type-aware compact views and retrieval addresses with explicit token/ratio compression budgets
src/pra_hf/large_record_index.py	Independent typed, BM25, and embedding indexes, bounded fusion, exact selectors, and lifecycle accounting
src/pra_hf/adaptive_context_runtime.py	Scoped backing, serializable token/byte and per-type ingestion policy, materialization, transport accounting, and cursors
src/pra_hf/progressive_context.py	Stable record views, native-index lifecycle audits, size-gated full backing, and lazy selected-region native encoding
src/pra_hf/external_memory.py	Resolver-neutral cold/warm/hot lifecycle and scoped caches
pra-hf-demo/pra_runtime_productization.ipynb	Executed user workflow

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[2] FlashML.org. Freetoken quickstart. `https://github.com/FlashML-org/FreeToken/blob/main/docs/quickstart.md`, 2026. Accessed 2026-08-27.

[3] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model

serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[6] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[7] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.